Cursuri – Seminarii – Labs OS

Lecture 1:

SH commands available on the system:

/bin/ls /bin | /usr/bin/grep -E sh | grep -E '^[a-z]{0,2}sh$' | sort | uniq

-/bin/ls /bin → Listeaza toate fisierele din directorul /bin

Exemplu de iesire: bash, sh, dash, ls, cat

| → Trimite controlul catre urmatoarea comanda

Grep -E sh → filtreaza doar liniile care contin sh: bash, sh, dash, ssh

| grep -E `^[a-z]{0,2}sh$ → Filtrarea mai stricta cu regex

^ → inceputul cuvantului

[a-z]{0,2} → 0 pana la 2 litere mici

Sh → urmat de sh

$ → sfarsitul cuvantului: sh, ash, csh, ksh, zsh

|sort → sorteaza alfabetic rezultatele

|uniq → elimina eventualele duplicate

Keyboard Shortcuts

Ctrl-S - Lock the console. You will be tempted to save your work with this, and then get confused.

Ctrl-Q - Unlock console. This is the antidote for Ctrl-S. Note that everything you typed while the console was locked will show up after you press this.

Ctrl-A - Jump to the beginning of the line
Ctrl-E - Jump to the end of the line

Ctrl-D - End of file when providing program input. When pressed on the first position of the command line, ends the connection (similar to running exit).

Ctrl-R - Search through the history of commands

Ctrl-K - Cut the text from the current position to the end of the line

Ctrl-Y - Paste what was cut with Ctrl-K

Ex comanda sort: ls | sort → sorteaza toate fisierele din directorul ls
Sort text.txt → sorteaza liniile unui fisier

cat text.txt | sort → sorteaza liniile unui fisier

Commands and Paths

ls → list the content of the current directory
cat text.txt → see the content of the file text.txt
ls -l → see the content of the current directory
ex: -rw-r--r-- 1 alexandra alexandra 153 Mar 11 16:45 a.c
-rw-r--r-- 1 alexandra alexandra 510 Mar 11 16:41 a.txt
Ls -l -a / ls -l -all → see the content of the current directory with the hidden files (ls -l -all is the long form for ls -l -a, they both have the same effect)
Mkdir abs → creating a new directory
Ls -l –color=never → display the content of the current directory, with all the details but without colors
--color=never → argument with value
Ls –help → prints the manual
Ls -l –color=never/labs
ls -l --color=never ~
total 12
drwxr-x--- 2 alexandra alexandra 4096 Mar 11 15:44 c-test-practice
-rw-r--r-- 1 alexandra alexandra 3066 Feb 23 07:35 c-test-practice.zip
drwxr-xr-x 2 alexandra alexandra 4096 Mar 11 19:57 labs

The UNIX System has a single root, unlike Windows, which has a root for every drive mounted (C:, D:)
The UNIX file system root is /
Ex: alexandra@ADRIANA-PC:~$ cd /
alexandra@ADRIANA-PC:/$ cd ~

pwd → command to find the path to the current directory
Ex: alexandra@ADRIANA-PC:~/labs$ pwd
/home/alexandra/labs

alexandra@ADRIANA-PC:~/labs$ cd /
alexandra@ADRIANA-PC:/$ pwd
/

alexandra@ADRIANA-PC:/$ cd ~
alexandra@ADRIANA-PC:~$ pwd
/home/alexandra

The Command Structure

- Space is the separator
- First word is the command
- Next words are the arguments

Exemplu short form vs long form
Short form: cut -d : -f 1, 2, 3 /etc/passwd
Long form: cut –delimiter=: --fields=1, 2, 3 /etc/passwd
Combined short forms: ps -e -f, ps -ef

Ls -l vs ls -all
ls -l

total 12
drwxr-x--- 2 alexandra alexandra 4096 Mar 11 15:44 c-test-practice
-rw-r--r-- 1 alexandra alexandra 3066 Feb 23 07:35 c-test-practice.zip
drwxr-xr-x 2 alexandra alexandra 4096 Mar 11 19:57 labs
alexandra@ADRIANA-PC:~$ ls -all
total 84
drwxr-x--- 8 alexandra alexandra 4096 Mar 11 19:57 .
drwxr-xr-x 3 root   root   4096 Feb 23 22:05 ..
-rw------- 1 alexandra alexandra 14900 Mar 11 17:42 .bash_history
-rw-r--r-- 1 alexandra alexandra   220 Feb 23 22:05 .bash_logout
-rw-r--r-- 1 alexandra alexandra 3771 Feb 23 22:05 .bashrc
drwx------ 2 alexandra alexandra 4096 Feb 23 22:05 .cache
drwxr-xr-x 2 alexandra alexandra 4096 Feb 28 12:37 .landscape
-rw------- 1 alexandra alexandra   20 Mar 11 11:08 .lesshst
drwx------ 4 alexandra alexandra 4096 Feb 24 21:57 .local
-rw-rw-r-- 1 alexandra alexandra   0 Mar 11 09:36 .motd_shown
-rw-r--r-- 1 alexandra alexandra   807 Feb 23 22:05 .profile
drwx------ 2 alexandra alexandra 4096 Mar 5 22:03 .ssh
-rw-r--r-- 1 alexandra alexandra   0 Feb 23 22:17 .sudo_as_admin_successful
-rw------- 1 alexandra alexandra 13538 Mar 11 19:57 .viminfo
drwxr-x--- 2 alexandra alexandra 4096 Mar 11 15:44 c-test-practice
-rw-r--r-- 1 alexandra alexandra 3066 Feb 23 07:35 c-test-practice.zip
drwxr-xr-x 2 alexandra alexandra 4096 Mar 11 19:57 labs

Recap: ls –al arata toate fisierele ascunse
-a → afiseaza toate fisierele, inclusive pe cele ascunse
-l → format detaliat
In Linux, fisierele ascunse incep cu .
. → directorul current
.. → directorul parinte

Ce fac liniile: cut -d : -f 1,2,3 /etc/passwd sau cut --delimiter=: --fields=1,2,3 /etc/passwd
O linie /etc/passwd arata asa: username:password:UID:GID:comment:home:shell
Dar, am extras doar username;password;UID
Ex: root:x:0 (root → numele utilizatorului, x → parola stocata criptat, 0 → UID, user id)
Ex: nume:x:1000
1 – 999 → conturi de system
1000+ → utilizatori normali
Pentru aliniere: awk -F: '{printf "%-15s %-6s %-20s %s\n", $1, $3, $6, $7}' /etc/passwd

ps -e -f sau ps -ef:
-e → arata toate procesele din system
-f → afiseaza formatul complet

Man → paginile de manual

Navigation: down → SPACE, up → b, search → /, exit → q

Finding the manual page you need: apropos keyword

alexandra@ADRIANA-PC:~$ apropos string

register_printf_modifier (3) - define custom behavior for printf-like fun...

PA_STRING (3const) - define custom behavior for printf-like functions

printf.h (3head) - define custom behavior for printf-like functions

printf_arginfo_size_function (3type) - define custom behavior for printf-...

PA_WSTRING (3const) - define custom behavior for printf-like functions

argz_stringify (3) - functions to handle an argz list

asprintf (3) - print to allocated string

atof (3) - convert a string to a double

atoi (3) - convert a string to an integer

atol (3) - convert a string to an integer

atoll (3) - convert a string to an integer

atoq (3) - convert a string to an integer

…

Whatis command:

Ex: alexandra@ADRIANA-PC:~$ whatis ls

ls (1) - list directory contents

alexandra@ADRIANA-PC:~$ whatis pwd

pwd (1) - print name of current/working directory

Connecting commands with Pipe |

| → takes the output of the command before it and passes it as input to the command after it

Ex: alexandra@ADRIANA-PC:~$ sort ~/labs/hello3.txt | head -n 10

Comenzi folosite: sort, uniq, grep, find, ps, who, last, wc

Find → cauta fisiere in directorul specificat dupa nume, tip marime

Ex: alexandra@ADRIANA-PC:~/labs$ find ~/labs -name "*.c"

/home/alexandra/labs/ex.c

/home/alexandra/labs/lab2_5.c

/home/alexandra/labs/lab2_3.c

/home/alexandra/labs/d.c

/home/alexandra/labs/g.c

/home/alexandra/labs/f.c

/home/alexandra/labs/main.c

/home/alexandra/labs/lab2_1.c

/home/alexandra/labs/lab2_6.c

/home/alexandra/labs/h.c

/home/alexandra/labs/c.c

/home/alexandra/labs/lab2_2.1.c

/home/alexandra/labs/lab2_2.c

/home/alexandra/labs/a.c

/home/alexandra/labs/lab2_4.c

/home/alexandra/labs/e.c

Ps → arata procesele care ruleaza pe sistem

Ex: ps -ef

Who → arata cine este conectat

alexandra@ADRIANA-PC:~/labs$ who

alexandra pts/1 2026-03-11 19:09

Last → arata istorucul login-urilor

alexandra@ADRIANA-PC:~/labs$ last

reboot system boot 6.6.87.2-microso Wed Mar 11 19:09 still running

reboot system boot 6.6.87.2-microso Wed Mar 11 16:15 - 19:07 (02:51)

Wc → numara linii, cuvinte, caractere

Wc -l → doar numarul de linii

Wc -w → doar numarul de cuvinte

Wc -c → doar numarul de caractere

alexandra@ADRIANA-PC:~/labs$ wc hello3.txt

42 352 1746 hello3.txt

42 de linii, 352 de cuvinte si 1746 caractere

Basics of C Programming in the Linux Command Line

- String vs Array → strings end with 0, arrays must be accompanied by their length

- &n is the adress of variable n
- *p is the content of pointer p

Int main (int argc, char **argv)

- Argv[0] → name of the command
- Argv[1] → is the first argument, argv[2] is the second argument, and so on
- Argc is the length of array argv

Memory

- Allocation → malloc
- Deallocation → free

Text Files

- All files are binary, but some of them contain only bytes from 0 to 127, which can be displayed as text, using the ASCII encoding. (other encodings: extended ASCII)

Regular Expressions

. → matches any single caracter

Ex: grep “a.c” text.txt: afiseaza liniile care contin a + orice caracter + c

abc

a1c

a-c

/ → Escape, changes the meaning of the character following it, between normal and special

Ex: grep “\.” text.txt: afiseaza liniile care contin punctul . (nu orice caracter)

[abc] → Matches any single character that appears in the list (in this case any lower-case letter)

Ex: grep “[abc]” text.txt: afiseaza liniile care contin a sau b sau c

[a-z] → Matches any single character that belongs to the range (in this case any lower-case letter)

Ex: grep “[a-z]” text.txt: afiseaza liniile care contin cel putin o litera mica

[^0-9] → Matches any single character that does not belong to the range (in this case anything that is not a digit)

Ex: grep “[0-9]” text.txt: afiseaza toate liniile care contin orice alt caracter in afara de cele din intervalul 0-9

^ → beginning of line

Ex: grep “^Ana” text.txt: afiseaza toate liniile care incep cu secventa “Ana”

$ → End of line

Ex: grep “mere$” text.txt: afiseaza liniile care se termina cu “mere”

Sau grep “mere.$” text.txt: afiseaza liniile care se termina cu mere + orice caracter

\< → beginning of word

Ex: exemplu.txt:

you

young

thank you

youtube

aboutyou

grep "\<you" exemplu.txt: afiseaza liniile care contin cuvinte care incep cu you

you

young

thank you

youtube

\> → End of word

grep "you\>" exemplu.txt: afiseaza liniile care contin cuvinte care se termina cu you

you

thank you

aboutyou

( ) → Groups several characters into an expression

Ex: grep -E “(ab)+” text.txt: afiseaza liniile care contin “ab” repetat de una sau mai multe ori

*→ Previous expression zero or more times

Ex: grep "ab*" text.txt: afiseaza liniile cu a urmat de 0 sau mai multe b

+ → Previous expression one or more times

Ex: grep -E "ab+" text.txt: afiseaza liniile cu a urmat de unul sau mai multe b

? → Previous expression zero or one times

Ex: grep -E "colou?r" text.txt: afiseaza liniile care contin: color, colour

{m, n} → previous expression at least m and at most n times

Ex: grep -E "a{2,4}" text.txt: afiseaza liniile care contin a repetat intre 2 si 4 ori

(Propozitii care contin secvente de genul: aa, aaa, aaaa)

| → Logical OR between parts of the regular expression

Ex: grep -E "cat|dog" text.txt: afiseaza liniile care contin “cat” sau “dog”

a. .* any sequence of characters
b. [a-zA-Z02468] → any letter, regardless of its case, and any even digit
c. [,] → space of comma
d. ^[^0-9]+$ → non-empty lines containing any characters except digits
e. ([Nn][Oo])+ → any refusal, no matter how insistent (eg No no no no no)

-E → use extended regular expressions

-v → display lines that do not match the given regular expression

-i → ignore upper/lower case when matching (match case-insensitively)

-q → Do not display the matching lines, just exit with 0 if not found, or 1 if not found

grep -E "^dan[0-9]*:" passwd.txt → include dan10, dan11, etc

grep -E "^dan([0-9])?:" passwd.txt → pana la dan9

grep -E "^[^0-9]+:" passwd.txt → afiseaza utilizatorii care nu au in username cifre

SED

Sed→ editor de flux pentru text.

Poate procesa fisiere fara sa le modifice direct.

Poate face cautari, inlocuiri, transliterari, stereri de linii, etc.

Substitute (s)

sed -E “s/regex/replacement/flags” a.txt

sed -E "s/foo/bar/g" a.txt

sed -E "s/(foo)(bar)/\2\1/" a.txt → cauta patternul foobar si il schimba in barfoo

Transliterare (y)

sed -E "y/characters/replacement/" a.txt

sed -E "y/abc/ABC/" a.txt

Stergerea liniilor care se potrivesc cu un regex (d)

sed -E “/regex/d” a.txt

sed -E "/^#/d" a.txt

sed 's/[^a-zA-Z0-9]//g' stateside.txt → lipirea tuturor characterelor

sed -E "s/[^aeiou]/*/gi" stateside.txt

sed -E "s/ //gi" stateside.txt

sed "/^$/d" stateside.txt

sed -E "/^[a-zA-Z0-9]*$/d" stateside.txt (sterge si liniile goale)

sed -E "/^[a-zA-sed Z0-9]+$/d" stateside.txt (nu sterge liniile goale)

sed -E "s/(face)(time)/\2\1/gi" stateside.txt → FaceTime devine TimeFace

sed -E "s/(face) (time)/\2 \1/gi" stateside.txt → Face Time devine Time Face

sed -E "s/([a-z])/(\1)/gi" stateside.txt → fiecare litera intre paranteze

sed -E "s/([a-z]+)/(\1)/gi" stateside.txt → fiecare cuvant intre paranteze

sed -E "s/([a-z]+)/\1 \1/gi" stateside.txt → dubleaza fiecare cuvant

Numeroteaza liniile: sed = lyrics.txt | sed 'N;s/\n/ /'

Afiseaza doar liniile pare: sed -n 'n;p' lyrics.txt

Afiseaza doar liniile impare: sed -n 'p;n' lyrics.txt

Inverseaza ordinea cuvintelor: sed -E ':a;s/^(([^]+)+)([^]+)$/\3 \1/;ta' lyrics.txt

sed "s/[aeiou]//gi" stateside.txt → elimina toate vocalele

sed -E "s/([a-z]+)/\U\1/i" stateside.txt → marcheaza cu majuscule

grep -Ei "\b[^aeiou]*([aeiou][^aeiou]*){2,}\b" stateside.txt → liniile care au cuvinte care contin cel putin 2 vocale

sed -E "y/elaoszbg/31405289/" /etc/passwd

```
sed -E "s/([aeiou]{3,})/(\1)/gi" /etc/passwd
```

AWK

Awk → procesare de text pe coloane
Fiecare linie = un rand din tabel, fiecare cuvant = o coloana
Awk 'selector {actiune}' fisier
Fisier: Ana are mere
Ion are pere
Comanda: awk '{ print $1 }' fisier.txt
Output: Ana
Ion

$1 → primul cuvant din fiecare linie
Campuri (coloane)
$0 → toata linia
$1 → primul cuvant
$2 → al doilea cuvant
$NF → ultimul cuvant
NF → numarul de cuvinte
NR → numarul liniei curente

awk '{ print $1 }' fisier.txt → primul cuvant de pe fiecare linie
awk '{ print NR, $0 }' fisier.txt → linia + numarul ei
awk 'NF > 3' fisier.txt → doar liniile cu mai mult de 3 cuvinte
awk '/are/' fisier.txt → doar liniile care contin „are"
awk '{ print $2 }' fisier.txt → al doilea cuvant
awk '{ print $NF }' fisier.txt → ultimul cuvant
awk 'BEGIN { print "Start" } { print $0 }' fisier.txt → BEGIN (ruleaza inainte)
awk '{ print $0 } END { print "Final" }' fisier.txt → END (ruleaza dupa)

awk -F ":" '{ print $1 }' /etc/passwd → schimbarea separatorului
awk -F ":" '{ print $1, $7 }' /etc/passwd → user + shell din /etc/passwd
awk -F ":" '$7 == "/bin/bash"' /etc/passwd → shell-ul este /bin/bash
awk 'END { print NR }' fisier.txt → numara liniile
awk '{ sum += $2 } END { print sum }' fisier.txt → suma numerelor din coloana 2

Display the username of all users having their last field end with "nologin"
i. awk -F: '$NF ~ /nologin$/ {print $1}' /etc/passwd
awk -F: '/nologin$/ {print $1, $7}' practice.txt
awk -F: '$NF ~ /\/bin\/bash/ {print $NF, $1}' passwd.txt ^C

Display the full names of all users having their username longer than 10 characters
i. awk -F: 'length($1) > 10 {print $5}' /etc/passwd
awk -F: '/^[^:]{11,}:/ {print $1, $5}' practice.txt

Provide the content of file prog.awk so that the command above will print all user on even line having a group id less than 20
Fara fisier: awk -F: '/^.+$/&& NR%2==0 && $3<700 {print N
R,$1,$3}' practice.txt

Cu fisier: In prog.awk
```
/^.+$/ && NR % 2 == 0 && $3 < 700 {
	print NR, $1, $3, $5
	}
```
In practice/:
awk -F: -f prog.awk practice.txt

awk -F: '$5 ~ /\<[A-Z]/ {print $5}' practice.txt | awk '{print $1}' → numele de familie al fiecarui utilizator

Sort → sorteaza liniile alfabetic
Uniq -c → elimina duplicatele consecutive, -c numara aparitiile fiecarui element
Sort -n -r → sorteaza numeric
→ foloseste prima coloana ca numar (-r → invers)
Head -n 10 → afiseaza primele 10 linii (cele mai frecvente 10 nume)

---

awk -F: '$5 ~ /\<[A-Z]/ {print $5}' practice.txt | awk '{print $1}' | sort | uniq -c | sort -n -r | head -n 10
awk '{print $1}' last.txt | sort | uniq -c | sort -n -r| head -n 20

last | awk '{print $1, $10}'
awk '{print $1, $10}' last.txt

Diferenta dintre:
last | sed -E "s/\(([0-9][0-9]:)/(0+/1:"
awlast | sed -E "s/\(([0-9][0-9]:)/(0+\1/"

sed -E "s/[():]/ /g" last.txt | awk '{print $1, ($12*60+$13)}' → pentru format de tip (hh:mm)

Timp total:
Program.awk
```
 {
	arr[$1] += $12*60+$13
	}
END{
	for (u in arr)
		print u, arr[u]
	}
```
sed -E "s/[():]/ /g" last1.txt | awk -f program.awk

```
Pentru sortare: {
	arr[$1] += $12*60+$13
	}
END{
	for (u in arr)
		print arr[u], u
	}

sed -E "s/[():]/ /g" last1.txt | awk -f program.awk | sort -n -r

Din command line: sed -E "s/[():]/ /g" last.txt | awk '{
	arr[$1] += $12*60+$13
	}
END{
	for (u in arr)
		print arr[u], u
	}'  | sort  -n -r
```

UNIX SHELL PROGRAMMING

Standard Input/Output
0 = standard input → „scanf”, „gets”, „cin”, „input”
1 = standard output → „printf”, „puts”, „cout”, „print”
2 = standard error

In Linux, totul este tratat ca un fisier (inclusiv terminalul)
Cand un program porneste are deja 3 fisiere, deschise automat
0 → stdin
1 → stdout
2 → stderr

ls > out.txt → salveaza output-ul intr-un fisier
ls /nu_exista 2> err.txt

2>&1 → trimite stderr (2) in acelasi loc unde merge stdout (1)
&1 → destinatia lui stdout

/dev/null → the file that contains nothing, and everything that you write to it dissapears. The Windows equivalent is NULL
➔ Used mainly to hide program output (either standard or error)

```
alexandra@LAPTOP-ALLE:~/practice$ gcc -Wall -Werror -g a.c -o a
alexandra@LAPTOP-ALLE:~/practice$ ./a <input.txt
Unsupported modealexandra@LAPTOP-ALLE:~/practice$ ./a lib <input.txt
**********
********
```

```
alexandra@LAPTOP-ALLE:~/practice$ ./a sys <input.txt
**********
********
alexandra@LAPTOP-ALLE:~/practice$ ./a sys <input.txt >output.txt
alexandra@LAPTOP-ALLE:~/practice$ cat output.txt
**********
********
alexandra@LAPTOP-ALLE:~/practice$
```

Pipe } → conecteaza doua programe
Redirect < → citeste direct din fisier

COMMAND TRUTH VARIABLES

Opposite of C:
0 → true
1 → false

test -f /etc/abc || echo "File /etc/abc does not exist"
test → verifica conditii in shell
-f /etc/abc → verifica daca fisierul /etc/ebc exista si este fisier regulat

cmd1 || cmd2 → executa cmd2 doar daca cmpd1 esueaza

```
alexandra@LAPTOP-ALLE:~/practice$ test -f a.c || echo "file a.c does not exist"
alexandra@LAPTOP-ALLE:~/practice$ test -f b.c || echo "file a.c does not exist"
file a.c does not exist
```

Test command conditional operators

String: ==, !=, -n, -z
== → egalitate
!= → inegalitate
-n → lungimea > 0 ($a nu este un string gol)
-z → lungimea = 0

```
a="abc"
b="abc"
[ "$a" == "$b" ] && echo Equal # Output: Equal

[ -n "$a" ] && echo Not empty # Output: Not empty

c=""
[ -z "$c" ] && echo Empty # Output: Empty
```

Integers: -lt, -le, -eq, -ne, -ge, -gt
-eq → egal (equal)

[5 -eq 5] → true
-ne → diferit (not equal)
[5 -ne 5] → true
-lt → less than (<)
[3 -lt 5] → true
-le → less or equal (<=)
[3 -le 3] → true
[5 -le 5] → true
-gt → greater than (>)
[5 -gt 3] → true
[7 -gt 3] → true
-ge → greater or equal (>=)
[7 -gt 5] → true
[5 -gt 5] → true

File system: -f, -d, -r, -w, -x
-f (FILE) → exista si este fisier regulat
[-f /etc/passwd] → true
-d (DIR) → exista si e director
[-d /etc] → true
-r (FILE) → fisierul e citibil
[-r /etc/passwd] → true
-w (FILE) → fisierul e scriibil
[-w myfile.txt] → true daca ai permisiuni
-x (FILE) → fisier executabil
[-x /bin/ls] → true

SHELL VARIABLES AND EMBEDDED COMMANDS

Variable Definition (Definirea variabilelor) → o variabila se defineste atunci cand spui compilatorului „Voi folosi un nume pentru a stoca un tip de date specific”.
Ex: char s[64]; //definim un sir de caractere cu 64 de pozitii
Char* p; //definim un pointer la char
Int i; //definim un intreg
Int k; //definim un intreg

Variable Reference (Referinta variabilelor) → O variabila este referita atunci cand o folosim in expresii sau apeluri de functii.
P = fgets(s, 64, stdin) //s si p sunt referite
For (int i = 0; i < strlen(s); i++) //i si s sunt referite
S[i] = ’*’
Fputs(s, stdout); //s este referita
K = read(0, s, 64); //k si s sunt referite
Write(1, s, k); //s si k sunt referite

Embedded Commands → Un embedded command (comanda incorporata) in shell inseamna o comanda care este rulata in interiorul unei alte comenzi sau expresii, iar rezultatul ei poate fi folosit imediat.
today = `date`
echo “Astazi este $today”

Embedded Commands

- Sunt puse traditional intre backsticks: `comanda`

```
today=`date`
echo "Astăzi este $today"
```

- Shell-ul executa comanda din backsticks si inlocuieste expresia cu iesirea comenzii

```
echo "Numărul de fișiere din director: `ls | wc -l`"
```

- Putem salva rezultatul unei comenzi intr-o variabila

```
N=`grep -E "/gr211/" /etc/passwd | wc -l`
echo "Numărul de utilizatori GR211: $N"

if [ $N -gt 0 ]; then
  echo "Există utilizatori GR211"
fi
```

Structura generala:

```
if [ conditie ]; then
    comenzi_daca conditia_este_adevarata
elif [ alta_conditie ]; then
    comenzi_daca_aceasta_conditie_este_adevarata
else
    comenzi_daca_niciuna_dintre_conditii_nu_este_adevarata
fi
```

```
if [ $N -gt 0 ]; then
> echo "Exista $N aparitii a lui 'girl' in stateside"
> else
> echo "Nu exista aparitii a lui 'girl' in stateside"
> fi
```

Exista 4 aparitii a lui 'girl' in stateside

A="Tom"

Referred as $A or ${A} a. echo $A is a human

b. echo $Acat is a feline or an application server - doesn't work

c. echo ${A}cat is a feline or an application server

> When used in strings delimited by ", variables and embedded commands will be replaced by their value. Strings delimited by ' do not allow any substitutions in their content.

alexandra@LAPTOP-ALLE:~/practice$ echo "AA is a GPS navigator"
TomTom is a GPS navigator
alexandra@LAPTOP-ALLE:~/practice$ echo 'AA is a GPS navigator'
AA is a GPS navigator

echo "There are $(grep -E -i "girl" stateside.txt | wc -l) aparitions of girl in stateside"

SHELL SCRIPTS

Script → un script este un fisier text care contine o serie de comenzi ce pot fi executate de un interpretor de comenzi (shell).

Exemple de shell-uri: bash, sh, zsh

Exemplu simplu de script Bash

#!/bin/bash # shebang – spune sistemului ce shell să folosească

echo "Hello, world!"

File execution permissions → In Linux/Unix, fiecare fisier are permisiuni care controleaza cine poate citi, scrie sau executa fisierul.

Permisiuni tipice:

r → citire

w → write

x → execute

Exemplu: ls -l a.c

-rw-r--r-- 1 alexandra alexandra 644 Mar 29 17:59 a.c

ls -l a

-rwxr-xr-x 1 alexandra alexandra 19624 Mar 29 18:01 a

Pentru a face un script executabil:

Chmod +x myscript.sh

Pentru a-l rula

./myscript.sh

Special Variables

Shell-ul defineste variabile speciale care ajuta la interactiunea cu scripturile:

$0 → Numele scriptului

$1, $2, ... → Primul, al doilea argument dat scriptului

$# → Numarul de argumente

$@ → Toate argumentele ca lista

$? → Codul de iesire al ultimei comenzi

$$ → PID-ul procesului scriptului

$! → PID-ul ultimei comenzi lansate in background

```
#!/bin/bash

echo "Script name: $0"

echo "First argument: $1"

echo "Total arguments: $#"
```

```
#!/bin/bash

# Afișează data și ora curentă

echo "Current date and time: $(date)"
```

```
#!/bin/bash

name=$1

echo "Hello, $name!"
```

```
#!/bin/bash

files_count=$(ls | wc -l)

echo "There are $files_count files in this directory."
```

1. Any text file wirh execution permissions can be a script, if it contains commands interpretable by the current shell
   Ex: text.txt
   #!/bin/bash
   echo "hello world"
   chmod +x test.txt
   ./test.txt
2. Comments start with #
3. Give the script execution permission: chmod 700 a.sh

alexandra@LAPTOP-ALLE:~$ ls -l

Permissions

Ls -l

total 164

-rw-r--r-- 1 alexandra alexandra 130091 Mar 29 15:31 last.txt

drwxr-xr-x 2 alexandra alexandra 4096 Apr 2 10:32 practice

-rw-r--r-- 1 alexandra alexandra 30810 Mar 29 15:31 ps.txt

The first character tells the type: - → Regular file, d→ directory

Characters 2-4 show the permission for the owner of the file

Characters 5-7 show the permission for the group of the file

Charactes 8-10 show the permission for everybody else

Each permission triplet describes the read, write and execution permissions

Can be described by number, by considering each of the 3 positions as a binary digit

i. 7 = 111 = rwx
ii. 6 = 110 = rw-
iii. 5 = 101 = r-w

Command chmod is used to assign permissions to files

chmod 700 a.sh gives the owner of the file full permission, and nothing to the group or the others

```
alexandra@LAPTOP-ALLE:~/practice$ cat shellpractice.sh

#!/bin/bash

echo "Name of the command: $0"

echo "Command line arguments: $1, $2, $3, $4, $5, $6, $7, $8, $9, ${10}, ${11}"

echo "All the arguments together: $*, $@"

echo "Number of command line arguments: $#"

echo "Exit code of previous command: $?"

alexandra@LAPTOP-ALLE:~/practice$ ./shellpractice.sh Ana are mere si pere si alune si gutui si piersici

Name of the command: ./shellpractice.sh

Command line arguments: Ana, are, mere, si, pere, si, alune, si, gutui, si, piersici

All the arguments together: Ana are mere si pere si alune si gutui si piersici, Ana are mere si pere si alune si gutui si piersici
```

```
Number of command line arguments: 11
Exit code of previous command: 0
```

```
alexandra@LAPTOP-ALLE:~/practice$ ./special_vars.sh a b c d e f g h i j k l m
Command: ./special_vars.sh
First four args: a b c d
All args: a b c d e f g h i j k l m
Arg count: 13
Command true exited with code 0
Command false exited with code 1
alexandra@LAPTOP-ALLE:~/practice$ cat special_vars.sh
#!/bin/bash
echo Command: $0
echo First four args: $1 $2 $3 $4
echo All args: $@
echo Arg count: $#
true
echo Command true exited with code $?
false
echo Command false exited with code $?
```

```
alexandra@LAPTOP-ALLE:~/practice$ cat using-shift.sh
#!/bin/bash
echo Command: $0
echo First four args: $1 $2 $3 $4
echo All args: $@
echo Arg count: $#
shift
echo Some args: $1 $2 $3 $4
echo All args: $@
```

```
echo Arg count: $#
shift
echo Some args: $1 $2 $3 $4
echo All args: $0
echo Arg count: $#
shift 3
echo Some args: $1 $2 $3 $4
echo All args: $@
echo Arg count: $#
alexandra@LAPTOP-ALLE:~/practice$ ./using-shift.sh a b c d e f g h i j k l m
Command: ./using-shift.sh
First four args: a b c d
All args: a b c d e f g h i j k l m
Arg count: 13
Some args: b c d e
All args: b c d e f g h i j k l m
Arg count: 12
Some args: c d e f
All args: ./using-shift.sh
Arg count: 11
Some args: f g h i
All args: f g h i j k l m
Arg count: 8
```

Unix Shell For Loop

The variables cycle through a list of space separated values

```
#!/bin/bash
for A in a b c d;
do
```

```
    echo Here is $A
done
for A in a b c d
do
    echo Here is $A
done
```

```
alexandra@LAPTOP-ALLE:~/practice$ cat for-args.sh
#!/bin/bash
for A in $@; do
    echo Arg A: $A
done
alexandra@LAPTOP-ALLE:~/practice$ ./for-args.sh ana are mere pere alune gutui si piersici
Arg A: ana
Arg A: are
Arg A: mere
Arg A: pere
Arg A: alune
Arg A: gutui
Arg A: si
Arg A: piersici
```

```
alexandra@LAPTOP-ALLE:~/practice$ cat for-args-2.sh
for A; do
    echo Arg A: $A
done
alexandra@LAPTOP-ALLE:~/practice$ ./for-args-2.sh ana are mere pere alune gutui si piersici
Arg A: ana
Arg A: are
```

```
Arg A: mere
Arg A: pere
Arg A: alune
Arg A: gutui
Arg A: si
Arg A: piersici
```

```
#!/bin/bash
S=0
for F in $1/*.c; do
N=`grep -E "[^ ]" $F | wc -l`
S=`expr $S + $N`
done
echo $S
```

Explicatie

#!/bin/bash → spune ca scriptul trebuie executat cu bash

For F in $1/*.c; do → daca rulam ./seminar2.sh practice atunci expresia devine echivalenta cu

For F in practice/*.c → toate fisierele .c din practice

N=1grep -E „[^]” $F | wc -l` → n primeste numarul de linii din fisierul F c care au cel putin un caracter diferit de spatiu

S=`expr $S + $N` → Aduna valoarea curenta din S cu N

Filename Wildcards

Similar but much simpler than regular expressions

Rules:

i. * → Mathces any sequence of characters, including a void sequence, but not the first dot in a filename
ii. ? → Matches any single character, but not the first dot in a filename
iii. [abc] → List of optional characters, support ranges like regular expressions
iv. [!abc] → Negated list of optional characters (similar to [^abc] from regular expressions)

```
#!/bin/bash
S=0
for F in `find $1 -type f -name "*.c"`; do
N=`grep -E "[^ ]" $F | wc -l`
S=`expr $S + $N`
done
echo $S
```

```
#!/bin/bash
S=0
while read F
do
   N=$(grep -E "[^ ]" "$F" | wc -l)
   S=$((S + N))
done < <(find "$1" -name "*.c")
echo $S
```

Find F in `find $1 -type f -mane „*.c”; dp → cauta toate fisierele .c in directorul $1

alexandra@LAPTOP-ALLE:~/practice$ find -type f -name "*.c"

./a.c

./b.c

alexandra@LAPTOP-ALLE:~$ find practice -type f -name "*.txt"

practice/stateside.txt

practice/lyrics.txt

practice/passwd.txt

. . .

alexandra@LAPTOP-ALLE:~$ find practice

practice

practice/p3.awk

practice/prog1.awk

. . .

Unix Shell IF/ELIF/ElSE/FI Statement

Every command is a condition

Commands can be grouped with parantheses and logical operators

a. Check whether a file does not exist or if it exists and is not readable
b. ! test -f a.txt || (test -f a.txt && ! test -r a.txt)

Recap: -q de la grep → do not display the matching lines, just exit with 0 if found, or 1 if not found. We will use this only later, when we get to Shell Programming

Exemplu folosire test:

alexandra@LAPTOP-ALLE:~$ test -f a.txt

alexandra@LAPTOP-ALLE:~$ echo $?

1

```
#!/bin/bash
for A in $@; do
	if test -f $A; then
		echo $A is a file
	elif test -d $A; then
		echo $A is a dir
	elif echo $A | grep -E -q "^[0-9]+$"; then
		echo $A is a natural number
	else
		echo We do not know what $A is
	fi
done
```

```
#!/bin/bash
for A in $@; do
	if [ -f $A ]; then
```

```
        echo $A is a file
    elif [ -d $a ]; then
        echo $A is a dir
    elif echo $A | grep -E -g "^[0-9]+$"; then
        echo $A is a number
    else
        echo We do not know what $A is
    fi
done
```

Unix Shell While Statement

Read user input until the input is stop

The user input is read with command read which stores the input in a variable given as argument

Script basic-while.sh

```
#!/bin/bash
while true; do
    read X
    if [ $X == stop ]; then → aici ar trebui ghilimele la $X si stop
        break
    fi
done
```

```
#!/bin/bash
while true; do
    read X
    if test "$X" == "stop"; then
        break
    fi
done
```

[. . .] este o forma alternativa pentru test

["$X" = "stop"]

test "$X" = "stop"

➔ Fac aceeasi verificare

```
#!/bin/bash
D=$1
S=$2
find $D -type f | while read F; do
N=`ls -l $F | awk '{print $5}'`
if test $N -gt $S; then
echo $F
fi
done
```

#!/bin/bash → linia „shebang”, ii spune sistemului de operare ca acest fisier trebuie executat folosing interpretorul Bash

find $D -type f | while read F; do

find $D -type f → Cauta toate fisierele -type f din directorul D

while read F → Parcurge lista linie cu linie, salvand calea fiecarui fisier in variabila F

N=`ls -l $F \ awk ’{print $5}’` → ls -l afiseaza detaliile fisierului si awk ’{print $5}’ extrage coloana a 5-a din rezultatul ls -l care reprezinta dimensiunea in bytes

Rezultatul este salvat in variabila N

bash large-files.sh /home/alexandra 1043

bash large-files.sh /home/alexandra ...

```
#!/bin/bash

F=""

while [ -z "$F" ] || [ ! -f "$F" ] || [ ! -r "$F" ]; do

read -p "Provide an existing and readable file path:" F

done
```

```
#!/bin/bash

F=""

while test -z "$F" || ! test -f "$F" || ! test -r "$F"; do

read -p "Provide an existing and readable file path:" F

done
```

Unix Processes

Process = program in execution.

Running a program multiple times => multiple processes

De ce pot rula „toate'' procesele daca avem putine CPU-uri?

→ Exista mai multe procese decat nuclee

Time-sharing = partajare in timp

→ CPU-ul da fiecarui proces un cuant de timp

SO imparte timpul CPU-ului in bucati mici → quanta de timp

Fiecare proces primeste pe rand CPU-ul pentru o perioada foarte scurta

Apoi este intrerupt si alt proces primeste CPU-ul → context switch

→ Quantele sunt foarte mici

De ordinul milisecundelor, ni se pare ca toate ruleaza simultan, dar fiecare proces progreseaza

Iluzite de paralelism numita concurenta (concurrency)

Problema: Race Condition

→ Mai multe procese acceseaza aceeasi resursa

→ Ordinea executiei conteaza

→ Rezultatul final devine imprevizibil

Ex: x = 0

Doua procese fac x = x + 1, de asteptat x = 2

Dar, se poate intampla:

Proces A citeste x = 0, proces B citeste x = 0

A scrie x = 1, B scrie x = 1 => rezultat final x = 1

#!/bin/bash F=$1 N=0 while [$N -lt 200]; do K=`cat $F` K=`expr $K + 1` echo $K > $F N=`expr $N + 1` done	#!/bin/bash echo 0 > a.txt # The script on the left is inc.sh ./inc.sh a.txt & ./inc.sh a.txt & ./inc.sh a.txt &

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <string.h>
int main(int argc, char** argv) {
	int f, k, i;
	f = open(argv[1], O_RDWR); //deschidere fisier
	if(argc > 2 && strcmp(argv[2], "reset") == 0) //reset
	{
		k = 0;
		write(f, &k, sizeof(int)); //se scrie 0
		close(f);
		return 0;
	}
	for(i=0; i<255*255; i++)
	{
		lseek(f, 0, SEEK_SET); //la inceputul fisierului
		read(f, &k, sizeof(int)); //citeste k
		k++; //incrementeaza
		lseek(f, 0, SEEK_SET); //la inceputul fisierului
		write(f, &k, sizeof(int)); //scriere k nou
	}
	close(f);
	return 0;
}
```

Creating processes in Unix

The UNIX way of creating a process is to duplicate the current process by making a copy of it using the system call fork => code will execute along split paths.

The code bellow will print ”after” twice/

That’s because after calling fork, there will be two processes: the original one (called parent), and the copy of it (called child). Both processes continue from the instruction after fork, and thus will both print ’’after’’.

```
#include<stdio.h>
#include<unistd.h>
int main(int argc, char** argv) {
        printf("before\n");
        fork();
        printf("after\n");
        return 0;
}
```

```
#include <stdio.h>
#include<unistd.h>
int main(int argc, char** argv) {
    int i;
    for (i = 0; i < 3; i++) {
        fork();
        printf("Din bucla\n");
    }
        printf("Din afara buclei\n");
        return 0;
}
```

1 proces

fork() => 2 procese

fork() => 4 procese

fork() => 8 procese

In total 7 child processes.

i = 0: 2 procese => doua afisari Din bucla

i = 1: 4 procese => 4 * Din bucla

i = 2: 8 procese => 8* Din bucla

2 + 4 + 8 = 14 * Din bucla

8 * Din afara buclei

What is the point of creating copies of the same process?

There is a way to distinguish between the parent and child processes, by the return value of fork(). It returns 0 in the child, and the PID of the child to the parent.

0 -> in the child

PID of the child -> in the parent

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main(int argc, char** argv)
{
```

```
    int r;
    r = fork(); //sistemul creeaza o copie exacta a procesului curent
    printf("PID=%d PPID=%d R=%d\n", getpid(), getppid(), r); //executata de doua ori, odata de parinte, odata de copil
    wait(0); //parintele se va opri aici si va astepta pana cand copilul isi termina executia
        //copilul va apela si el wait(0), dar deoarece nu are proprii copii de asteptat, apelul va returna imediat o eroare (pe care programul o ignora) si se va inchide
    return 0;
}
```

R → valoarea de retur a functiei fork()

PID=1030 PPID=309 R=1031 //parinte

PID=1031 PPID=1030 R=0 //copil

This allows us to execute different code in the child, than in the parent. The exit call in the child ensures that the child is not going to continue with the parent code after finishing the if-statement.

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
#include<sys/types.h>
#include<sys/wait.h>
int main(int argc, char** argv) {
int pid;
pid = fork();
if(pid == 0) {
printf("Child-only code\n");
exit(0);
}
printf("Parent-only code\n");
wait(0);
return 0;
}
```

This allows us to write concurrent programs, like a server that serves multiple requests simultaneously. A schematic sequential server code would look like the code bellow.

If Google were implemented like this, any search you make would have to wait unli all the other searches done by other people in the world before you, are completed.

```
while(1) {

req = get_request();  //serverul asteapta sa vina o cerere

res = process_request(req);  //serverul proceseaza cererea

send_response(res);  //serverul trimite raspunsul inapoi

}
```

In timp ce serverul este la pasul 2 (procesare), el este „blocat”.

Cum rezolva fork() aceasta problema?

```
while(1) {
        req = get_request();
        if (fork() == 0) {
                //suntem in copil
                res = process_request(req);
                send_response(res);
                exit(0);
        }
}
// parintele merge imediat inapoi la inceputul buclei (while) pentru a primi urmatoarea cerere, fara sa astepte dupa copil
```

We can now write code like this. Every request is served by a separate process, and the main server (the parent process) can get a new request immediately and spawn another child process to serve it concurrently.

Zombie Processes

It is natural that a parent process be interested in the execution status (exit code) of a child process it created). To get that state, the parent process uses either the wait or waitpid system calls.

What if the child process ends before the parent calls wait? The system keeps the child in a zombie state: it doesn’t execute anything, but it appears in the list of processes.

A zombie process stops when the parent calls wait or whatpid.

Zombie processes are problematic because if they are not ”waited” they will grow in number, occupying PIDs and bringing the system to a point where no other processes can be created because of lack of PIDs.

<defunct> → Proces Zombie

In Unix, cand un proces copil se termina (prin exit sau la finalul functiei main), el nu dispare instantaneu de tot. Sistemul de operare pastreaza o mica parte din el (codul de terminare si ID-ul procesului) pentru ca parintele sa poata afla cum s-a terminat executia acelui copil.

Procesul este mort: Nu mai consuma memorie RAM, nu mai foloseste CPU.

Procesul este defunct: Exista inca o intrare in tabela de procese a nucleului (kernel).

Un proces ramane in starea <defunct> pana cand parintele sau apeleaza functia wait().

In momentul in care parintele executa wait(), el citeste starea de iesire a copilului.

Abia atunci, kernel-ul sterge definitiv intrarea procesului in tabela.

Copilul moare la secunda 10.

Parintele insa doarme pana la secunda 15 (primul sleep(15)).

Intre secunda 10 si secunda 15, la comanda ps -ef, copilul este marcat cu <defunct>, pentru ca el a murit, dar parintele inca nu s-a trezit ca sa dea wait().

The system call wait waits for any process to end. If you want to wait for a a specific one, use whatpid.

The argument for wait is a pointer to int, where it will return the child process exit code. But since we do not care for that value, we just pass a zero (NULL).

Daca vrem sa stim ce s-a intamplat cu copilul:

```
int status;
pid_t child_pid = wait(&status);
if (WIFEXITED(status)) {
    printf("Copilul %d s-a terminat normal cu codul: %d\n", child_pid, WEXITSTATUS(status));
} else if (WIFSIGNALED(status)) {
    printf("Copilul a fost omorât de un semnal (ex: CTRL+C)!\n");
}
```

Forma standard: wait(NULL);

Daca vrem sa asteptam un copil anume: waitpid(pid, &status, 0);

The correct way to create a child process must involve calls to fork, exit and wait.

1. Creearea unui singur copil

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc, char** argv){
    int pid = fork();
    if (pid == 0) {
        //copilul executa acest bloc
        printf("Process created\n");
        exit(0);
    }
    wait(0); //parintele asteapta singurul copil creat
}
```

2. Crearea mai multor procese (Modelul „Fan-out”)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc, char** argv){
    for (int i = 0; i < 3; i++) {
        int pid = fork();
        if (pid == 0) {
            printf("Process created\n");
            //fflush(stdout);
            exit(0);
        }
    }
```

```
	for (int i = 0; i < 3; i++) {
		wait(0);
	}
}
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main(int argc, char** argv) {
	for (int i = 0; i < 3; i++) {
		if (fork() == 0) {
			printf("%d %d\n", getppid(), getpid());
			for (int j = 0; j < 3; j++) {
				if (fork() == 0){
					printf("%d %d\n", getppid(), getpid());
					exit(0);
				}
			}
			for (int j = 0; j < 3; j++) {
				wait(0);
			}
			exit(0);
		}
	}
	for(int i = 0; i < 3; i++) {
		wait(0);
	}
```

```
    printf("%d %d\n", getppid(), getpid());
    return 0;
}
```

Output:

```
618 619
618 620
618 621
619 622
620 623
620 626
621 625
619 628
620 629
621 627
619 624
621 630
328 618
```

Unix Signals

There is no wait called in the concurrent server code above. That is because we don't have any good place to call it, without knowing about signals. The solution bellow makes the entire server sequential. Calling wait after the infinite loop will never be reached, so it is pointless.

```
while(1) {
        req = get_request();
        pid = fork();
        if(pid == 0) {
            res = process_request(req);
            send_response(res);
            exit(0);
        }
        wait(0); // all is sequential now :-(
```

}

Signals are mechanisms that interrupt a process execution and determine it to run a handler (usually a default one) associated with the signal number.

Ciclul unui semnal:

1. Generarea (Generation) → Un eveniment declanseaza semnalul. Acesta poate proveni din exterior (ex apasarea Ctrl + C genereaza SIGINT), de la hardware (ex: impartirea la 0 genereaza SIGFPE), sau de la un alt proces.
2. Livrarea (Delivery) → Sistemul de operare transmite semnalul catre procesul tinta, modificandu-i starea curenta pentru a indica faptul ca semnalul este primit.
3. Manevrarea (Handling) → Procesul reactioneaza la semnal prin una dintre cele trei optiuni:

a. Actiunea implicita (Default): Comportamentul nativ (cum ar fi terminarea fortata a procesului sau crearea unui fisier core dump)
b. Ignorarea (Ignore): Procesul ignora complet semnalul (cu exceptia SIGKILL si SIGSTOP, care nu pot fi ignorate sau capturate).
c. Capturarea (Catching): Procesul intrerupe executia normala si trece la un gestionare (handler) personalizat pe care l-ai definit in cod.

Exemple de semnale:

1. SIGINT(2) → Ctrl + C
2. SIGSEV(11) → Segment Violation → procesul incearca sa acceseze o zona de memorie nepermisa
3. SIGCHLD(17 sau 20 in functie de arhitectura) → Un copil isi incheie executia

When a Putty session is closed rudely (from the X button), all processes in the session receive signal number 1 (SIGHUP – hangup) whose default behaviour is also to stop the process.

A process can assign a custom function to be executed when a certain signal is received.

A program that refuses to stop when it receives Ctrl – C.

```
#include <stdio.h>

#include <signal.h>

void f(int sgn) { //handler-ul sau gestionarul de semnal

    printf("I refuse to stop!\n"); //in loc sa se termine, programul va apela aceasta functie cand primeste semnalul si va afisa acest mesaj

}

int main(int argc, char** argv) {

    signal(SIGINT, f);

    while(1); //o bucla infinita care mentine procesul in viata, permitandu-i sa astepte si sa receptioneze semnale
```

```
        return 0;
    }
```

The signal system call does not signal anything, it only registers a function to be called when the signal is received. To send a signal to a process use the system call kill (don't confuse it with the command havind the same name).

Ex: Trimiterea unui semnal cu kill

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
void handle_signal(int sig) {
    printf("Copilul a primit semnalul %d\n", sig);
}
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        //Procesul copil
        signal(SIGUSR1, handle_signal);
        while (1) {
            sleep(1);
        }
    }
    else {
        //Procesul parinte
        sleep(1);
        printf("Parintele trimite semnalul SIGUSR1 catre fiu (PID: %d)\n", pid);
        kill(pid, SIGUSR1);
        sleep(1);
```

```
        printf("Parintele trimite SIGKILL catre fiu (PID: %d)\n", pid);
        kill(pid, SIGKILL);
        wait(NULL);
    }
    return 0;
}
```

Output:

Parintele trimite semnalul SIGUSR1 catre fiu (PID: 1165)

Copilul a primit semnalul 10

Parintele trimite SIGKILL catre fiu (PID: 1165)

So how do we use signals to solve the concurrent server zombie problem? When a child process stops, the parent receives signal SIGCHLD. We cand register a function to SIGCHLD that simply calls wait, and thus the child process stops being a zombie right away.

```
#include <stdio.h>
#include <signal.h>
void f(int sgn) {
    wait(0);
}
int main(int argc, char** argv) {
    signal(SIGCHLD, f);
    while(1) {
        req = get_request();
        pid = fork();
        if (pid == 0) {
            res = process_request(req);
            send_response(res);
            exit(0);
        }
    }
    return 0;
}
```

Procesul parinte continua sa execute bucla while(1) si sa preia cereri noi fara sa astepte finalizarea copiilor. Cand un proces copil isi termina sarcina si se inchide, sistemul de operare trimite automat semnalul SIGCHLD catre parinte.

signal(SIGCHLD, f); → instructiune in limbajul C care inregistreaza un gestionar (handler) pentru un anumit semnal. Ii transmite sistemului de operare cum sa reactioneze cand procesul curent (parintele) primeste semnalul SIGCHLD.

SIGNAL → functia din sistem care configureaza comportamentul procesului la primirea unui semnal specific.

An even simpler method is to ignore the SIGCHLD signal which causes the system to immediately kill the child process and not turn it into a zombie.

```
#include <stdio.h>

#include <signal.h>

int main(int argc, char** argv) {

    signal(SIGCHLD, SIG_IGN);

    while(1) {

        req = get_request();

        pid = fork();

        if (pid == 0) {

            res = process_request(req);

            send_response(res);

            exit(0);

        }

    }

    return 0;

}
```

signal(SIGCHLD, SIG_IGN) → Cand un proces copil se termina, sistemul de operare stie automat sa ii elibereze resursele si sa il stearga din tabela de procese. Parintele nu mai este nevoit sa intervina cu wait() sau whatpid().

Biblioteci care trebuie adaugate: #include <stdio.h>, #include <stdlib.h>, #include <signal.h>, #include <unistd.h>

1. Implement a program that upon receiving SIGINT (Ctrl-C) asks the user if he/she is sure the program should stop, and if the answer is yes, it stops, otherwise it continues.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>
void f(int sgn) {
	char s[32];
	printf("Are you sure you want me to stop [y/N]?\n");
	scanf("%s", s);
	if (strcmp(s, "y") == 0) {
		exit(0);
	}
	else {
	printf("Continuing the execution of the program\n");
	}
}
int main(int argc, char** argv) {
	signal(SIGINT, f);
	while(1);
	return 0;
}
```

Write a program that creates three child processes that just loop forever. When the parent process gets SIGUSR1 it sends SIGUSR2 to the child processes. When the parent process gets SIGUSR2, it sends SIGKILL to the child processes. The child processes should report the signals received. Note in the solution below that the child process does not override the SIGKILL handler, as it will not have any effect.

Acest program ilustreaza comunicarea dintre un proces parinte si trei procese copil prin intermediul semnalelor (signals). Parintele preia doua semnale diferite si, in functie de cel primit, trimite comenzi ulterioare copiilor.

sgn == SIGUSR1 ➔ Parintele afiseaza PID-ul sau si trimite SIGUSR2 catre toti copiii din vector

sgn == SIGUSR2 ➔ Parintele afiseaza PID-ul sau si trimite SIGKILL catre toti copiii din vector

```
#include <stdlib.h>
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>
#include <signal.h>
int children[3];
void hp(int sgn)
{
	if (sgn == SIGUSR1) {
		printf("%d: sending SIGUSR2\n", getpid());
		for (int i = 0; i < 3; i++) {
			kill(children[i], SIGUSR2);
		}
	}
	else if (sgn == SIGUSR2) {
		printf("%d: sending SIGKILL\n", getpid());
		for (int i = 0; i < 3; i++) {
			kill(children[i], SIGKILL);
		}
	}
}
void hc(int sgn)
{
	if (sgn == SIGUSR2) {
		printf("%d: Received SIGUSR2\n", getpid());
	}
}
int main(int argc, char** argv) {
```

```
    printf("PID-ul parintelui este: %d\n", getpid());
    signal(SIGUSR1, hp);
    signal(SIGUSR2, hp);
    for (int i = 0; i < 3; i++)
    {
        children[i] = fork();
        if (children[i] == 0) {
            signal(SIGUSR2, hc);
            while(1);
            exit(0);
          }
    }
    wait(0); wait(0); wait(0);
    return 0;
}
```

```
alexandra@LAPTOP-ALLE:~$ kill -SIGUSR1 1996
alexandra@LAPTOP-ALLE:~$ kill -SIGUSR2 1996
```

```
PID-ul parintelui este: 1996
1996: sending SIGUSR2
1997: Received SIGUSR2
1998: Received SIGUSR2
1999: Received SIGUSR2
1996: sending SIGKILL
```

Running other programs using the Exec System Calls

When you want to run a new program from an existing process, the exec functions replace the current process image with a new process image.

The exec Function Suffixes:

-l → list: the arguments are passed as a list of strings

-v → vector: the arguments are passed as an array (vector) of pointers to strings. The last element of the array must be NULL.

-p → path: uses the PATH environment variable to find the program name. You don't need to specify the absolute path (e.q. just 'grep' instead of '/bin/grep')/

-e → environment: Allows you to pass an array of pointers to environment variables

Examples:

```
char* a[] = {"grep", "-E", "/an1/gr911/", "/etc/passwd", NULL};

execvp("grep", a);

char* a[] = {"/bin/grep", "-E", "/an1/gr911/", "/etc/passwd", NULL};

execv("/bin/grep", a);

execlp("grep", "grep", "-E", "/an1/gr911/", "/etc/passwd", NULL);

execl("/bin/grep", "/bin/grep", "-E", "/an1/gr911/", "/etc/passwd", NULL);
```

1. Functia EXECVP

- Preia argumentele sub forma unui vector (array) de tip char* si cauta programul prin variabila de mediu PATH (nu necesita calea absoluta).

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

int main() {

    pid_t pid = fork();

    if (pid < 0) {

        perror("Eroare la fork");

        exit(EXIT_FAILURE);

    }

    else if (pid == 0) {

        //procesul copil

        //argumentele sunt un vector terminat cu NULL

        char* args[] = {"ls", "-l", NULL};

        //cauta ls in PATH

        execvp("ls", args); //echivalent cu ls -l sau /bin/ls -l
```

```
            perror("Eroare la execvp");
            exit(EXIT_FAILURE);
        }
        else {
            wait(NULL);
            printf("Procesul copil s-a incheiat (execvp).\n");
        }
        return 0;
}
```

2. Functia EXECL

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
        pid_t pid = fork();
        if (pid < 0) {
            perror("Eroare la fork");
            exit(EXIT_FAILURE);
        } else if (pid == 0) {
            //procesul copil
            printf("Copilul ruleaza comanda folosind execl\n");
            execl("/bin/ls", "/bin/ls", "-l", NULL);

            //daca execl reuseste, aceasta linie nu se mai executa
            perror("Eroare la execl");
            exit(EXIT_FAILURE);
        } else {
            wait(NULL);
            printf("Procesul parinte a terminat\n");
```

```
    }
    return 0;
}
```

3. Functia EXECV

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("Eroare la fork");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) {
        printf("Copilul ruleaza comanda folosind execv\n");

        char* args[] = {"/bin/ls", "-l", NULL};
        execv("/bin/ls", args);

        perror("Eroare la execv");
        exit(EXIT_FAILURE);
    }
    else {
        wait(NULL);
        printf("Procesul parinte a terminat\n");
    }
    return 0;
}
```

4. Functia EXECLP

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("Eroare la fork");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) {
        printf("Copilul ruleaza comanda folosind execlp\n");
        execlp("ls", "ls", "-l", NULL);

        perror("Eroare la execlp");
        exit(EXIT_FAILURE);
    }
    else {
        wait(NULL);
        printf("Procesul parinte a terminat\n");
    }
    return 0;
}
```

What exactly is the PATH? A UNIX Shell variable that contains paths where the Shell should look for the program you run, unless you specify an absolute or relative path like ./myprog

How PATH works in practice? When you open your terminal and type a command such as ls, grep or gcc, you don't usually type the full path of the executable (like /bin/ls or /usr/bin/gcc).

Instead, the shell takes the name you typed and consults the PATH variable to find the file.

If it searches all directories in the PATH and doesn't find the program, it returns an error (e.g. command not found).

Unexpected behaviour: the exec system calls re-uses the current process to run the other program. Essentially they wipe out the current code, and replace it with the code of the new program. If the exec call fails (usually due to the program not being found) the calling process continues to execute.

```
#include <stdio.h>

#include <unistd.h>

int main(int argc, char** argv) {

    execlp("grep", "grep", "-E", "alexandra", "/etc/passwd", NULL);

    printf("If grep is in the PATH, then execlp succeeds, and this will never be printed.\n");

    return 0;

}
```

As the first argument of a program (argument 0) is always the command name, we need to pass that argument explicitly, hence the duplication of the command name.

The last NULL argument is required in order to mark the end of the arguments.

If the exec system calls override the current process, how can I run another program and still keep my process? Simple, just create a child process and run exec in it.

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

int main(int argc, char** argv) {

    int pid = fork();

    if (pid == 0) {

        execlp("grep", "grep", "-E", "alexandra", "/etc/passwd", NULL);

        exit(1);

    }

    //do some other work in the parent

    //printf("Hello World\n");

    wait(0);

    printf("Hello World\n");
```

```
    return 0;
}
```

Although the exec call overwrites the child process code, we still want to keep the exit call there, just in case execlp fails, thus preventing the child process from executing parent code.

1. Write a C program that measures the duration of another programs execution given as command line argument along with its own arguments (i.e. ./measure grep -E "/an1/gr911/" /etc/passwd). In solving this problem, we will rely on the fact that although argc gives us the length of argv, argv also has an extra element with the value NULL (i.e. argv[argc] == NULL). In other words, argc is redundant, yet comfortable to use. The ending NULL of argv, allows as to pass it directly to execvp.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/time.h>
int main(int argc, char** argv) {
    struct timeval t0, t1;
    double duration;
    int status;
    if (argc < 2) {
        fprintf(stderr, "No command provided.\n");
        exit(1);
    }
    gettimeofday(&t0, NULL);
    if (fork() == 0) {
        execvp(argv[1], argv + 1);
        exit(1);
    }
    wait(&status);
    gettimeofday(&t1, NULL);
    duration = ((t1.tv_sec - t0.tv_sec) * 1000.0 + (t1.tv_usec - t0.tv_usec)/1000.0)/1000.0;
```

```
    printf("Duration: %lf seconds\n", duration);
    //tv_sec -> secunde
    //tv_usec -> microsecunde
    return WEXITSTATUS(status);
}
```

1. Write a program that gets programs as command line arguments, and executes them using fork and exec. In case exec fails or the program it executes exits with an error code, report the relevant details standard error. Notice in the solution below, the usage of strerror, errno, wait and WEXITSTATUS in the solution above. Read the UNIX manual to understand what they do.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <string.h>
#include <errno.h>
int main(int argc, char** argv) {
    int status;
    for (int i = 1; i < argc; i++) {
        if (fork() == 0) {
            if (execlp(argv[i], argv[i], NULL) == -1) {
                fprintf(stderr, "Failed to execute program \"%s\" : %s\n", argv[i], strerror(errno));
                //strerror(errno) -> Transforma codul numeric intr-un mesaj text
                exit(1);
            }
        }
        wait(&status);
        if (WEXITSTATUS(status) != 0) {
```

```
                fprintf(stderr, "Program \"%s\" failed with exit code %d\n", argv[i],
WEXITSTATUS(status));
            }
        }
        return 0;
}
alexandra@LAPTOP-ALLE:~/processes$ ./ex17 ls pwd fakecommand
a.txt   ex11.c  ex15.c  ex3    ex7.c  prog     prog3    prog6.c  run.sh
ex      ex12    ex16    ex3.c  ex8    prog.c   prog3.c  prog7    runn.sh
ex.c    ex12.c  ex16.c  ex4.c  ex8.c  prog1    prog4    prog7.c  signals4.c
ex1.c   ex13    ex17    ex5.c  ex9    prog1.c  prog4.c  prog8
ex10    ex13.c  ex17.c  ex6    ex9.c  prog1.sh  prog5    prog8.c
ex10.c  ex14    ex2     ex6.c  inc    prog2    prog5.c  prog9
ex11    ex14.c  ex2.c   ex7    inc.c  prog2.c  prog6    prog9.c
/home/alexandra/processes
Failed to execute program "fakecommand" : No such file or directory
Program "fakecommand" failed with exit code 1
```

Unix Pipe Communication

Suppose we want to speed-up a calculation by parallelizing it using processes.

Parallel addition of a vector's elements

The program bellow is a very simple implementation of the idea above but the rezult will be 6 instead of 10.

Why? The child process stores a[2] in its own memory, and the parent process use the a[2] from its own memory, which does not have the child's calculation.

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc, char** argv) {
    int a[4] = {1, 2, 3, 4};
    int pid;
```

```
    pid = fork();
    if (pid == 0) {
        a[2] = a[2] + a[3];
        exit(0);
    }
    a[0] = a[0] + a[1];
    wait(0);
    a[0] = a[0] + a[2];
    printf("%d\n", a[0]);
    return 0;
}
```

To fix this we need a way of communicating between processes. The easiest way to do this is using pipes.

A pipe is a buffer in memory that is opened like a file, twice, once for reading and once for writing. The opening is done automatically when we create it and we get back an array with the two int descriptors that result. The descriptor on position 0 is for reading, and the one on position 1 is for writing.

Array with two int descriptors

- ➔ 0 (reading)
- ➔ 1 (writing)

Special read behaviours

i. Removes data from the pipe
ii. If the pipe is empty, it waits for some data to arrive or for no writer to be connected to the pipe
iii. It may not return all the data requested to be read, but it returns the length of what is actually read.

Special write behaviours

i. Adds data to the pipe
ii. If the pipe is full, it waits for some space to be made or for no reader to be connected to the pipe
iii. It may not write all the data given, but it returns the length of what it actually wrote.

Behaviours ii. above cand lead to processes getting stuck on pipe operations. To avoid such situations, make sure you close each pipe end (the int descriptors) as soon as it is not needed anymore.

It is inherited from parent to child, this being the only mechanism of sharing it. Hence, in order for two processes to communicate through a pipe, one must inherit from the other, or both must inherit it from a common ancestor.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main(int argc, char** argv) {
    int a[4] = {1, 2, 3, 4};
    int pid, p[2];
    pipe(p);
    //se creeaza conducta (pipe) prin pipe(p)
    //aceasta returneaza doi descriptori de fisier in vectorul p
    //p[0] --> capatul de citire
    //p[1] --> capatul de scriere
    pid = fork(); //sistemul creeaza un proces copil care este o copie a procesului parinte
    if (pid == 0) { //pentru copil pid = 0
        close(p[0]); //se inchide capatul de citire, pentru ca procesul nu va citi, ci doar va scrie in conducta
        a[2] += a[3]; //a[2] = 3 + 4 = 7
        write(p[1], &a[2], sizeof(int)); //se scrie valoarea 7 in capatul de scriere al pipe-ului
        close(p[1]); //se inchide capatul de scriere dupa utilizare
        exit(0); //procesul copil isi incheie executia cu succes
    }
    close(p[1]); //parintele inchide capatul de scriere, deoarece el doar va citi din pipe
    a[0] += a[1]; //a[0] = 1 + 2 = 3
    read(p[0], &a[2], sizeof(int)); //parintele citeste din pipe si suprascrie valoarea din a[2] cu cea primita (adica 7)
    close(p[0]); //inchide capatul de citire dupa ce a terminat de primit date
    wait(0); //parintele se blocheaza si asteapta ca procesul copil sa isi incheie executia
    a[0] += a[2]; //se aduna cele doua valori modificate: 3 + 7 = 10
    printf("%d\n", a[0]); //afiseaza rezultatul final, adica 10
    return 0;
```

}

What would happen if we move close(p[1]) between lines pipe(p) and pid = fork()?

Child inherits a closed pipe end, namely p[1].

Daca am plasa wait(0) inainte de read, comportamentul programului ar fi identic din punct de vedere al rezultatului final. Cu toate acestea, din punct de vedere al eficientei, este preferat sa lasi procesul parinte sa preia datele din pipe cat mai repede, pentru a evita umplerea buffer-ului conductei, iar mai apoi sa colectezi procesul copil.

Advice: use a separate pipe for each direction of communication.

Attention: do not confuse a pipe with its ends; there are always two ends to a pipe.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
int main(int argc, char** argv) {
    int a[4] = {1, 2, 3, 4};
    int pid, p[2];
    pipe(p);
    pid = fork();
    if (pid == 0) {
        close(p[0]);
        a[2] += a[3];
        write(p[1], &a[2], sizeof(int));
        close(p[1]);
        exit(0);
    }
    wait(0);
    close(p[1]);
    a[0] += a[1];
    read(p[0], &a[2], sizeof(int));
    close(p[0]);
```

```
    a[0] += a[2];
    printf("%d\n", a[0]);
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
int main(int argc, char** argv)
{
    int p2a[2], a2b[2], b2p[2], n;
    //p2a -> trimite date de la parinte catre procesul A
    //a2b -> trimite date de la procesul A catre procesul B
    //b2p -> trimite date de la procesul B catre parinte
    pipe(p2a); pipe(a2b); pipe(b2p);

    if (fork() == 0) { //A
    //citeste din p2a[0] de la parinte
    //scrie in a2b[1] catre B
        close(p2a[1]); //inchide scrierea de la parinte la A
        close(a2b[0]); //inchide citirea de la A la B
        close(b2p[0]); //procese care nu au legatura cu A
        close(b2p[1]); //procese care nu au legatura cu A
        while(1) {
            if (read(p2a[0], &n, sizeof(int)) <= 0) {
                break;
            }
```

```
            if (n <= 0) {
                break;
            }
            printf("A: %d -> %d\n", n, n-1);
            n--;
            write(a2b[1], &n, sizeof(int));
        }
        close(p2a[0]);
        close(a2b[1]);
        exit(0);
    }
if (fork() == 0) { //B
    //citeste din a2b[0]
    //scrie in b2p[1]
        close(p2a[0]);
        close(p2a[1]);
        close(a2b[1]);
        close(b2p[0]);
        while(1) {
            if (read(a2b[0], &n, sizeof(int)) <= 0) {
                break;
            }
            if (n <= 0) {
                break;
            }
            printf("B: %d -> %d\n", n, n-1);
            n--;
            write(b2p[1], &n, sizeof(int));
        }
        close(a2b[0]);
```

```
            close(b2p[1]);
            exit(0);
        }
    //citeste din b2p[0]
        //scrie in p2a[1]
        close(a2b[0]);
        close(a2b[1]);
        close(b2p[1]);
        close(p2a[0]);

        n = 20;
        write(p2a[1], &n, sizeof(int));

        while(1) {
            if (read(b2p[0], &n, sizeof(int)) <= 0) {
                break;
            }
            if (n <= 0) {
                break;
            }
            printf("P: %d -> %d\n", n, n-1);
            n--;
            write(p2a[1], &n, sizeof(int));
        }
        close(b2p[0]);
        close(p2a[1]);
        wait(0);
        wait(0);
        return 0;
}
```

Notice how many closw function calls we have. We always close all unnecessary pipe ends as soon as possible.

Mutarea instructiunilor close la sfarsitul buclei while schimba modul in care sistemul de operare gestioneaza resursele si operatiunile de intrare/iesire, ceea ce duce la un blocaj (cunoscut si ca deadlock) sau o asteptare infinita.

Intarzierea inchiderii capetelor de scriere

Atunci cand nu inchizi capetele de scriere (cum ar fi p2a[1] de la inceput), descriptorul ramane deschis. Functia read din procesul A blocheaza executia si asteapta date. Sistemul de operare considera ca procesul A ar mai putea scrie in acea conducta, asa ca read nu va returna niciodata valoarea 0 (care semnaleaza terminarea fluxului EOF). Acest lucru face ca procesul sa ramana blocat, asteptand la nesfarsit.

Interferenta cu alte conducte → procesul A pastreaza deschise capetele de citire/scriere din conducta b2p pe toata durata buclei while.

Afisarea se produce deoarece procesele trimit mesaje corect in bucla pana la un moment dat, dar cand se goleste conducta si se asteapta o noua valoare, procesul se blocheaza in read.

Recap

Special behaviour of read and write when working with pipes

La lucrul cu pipe-uri (conducte anonime), functiile read si write au un comportament specific care le diferentiaza de operatiunile pe fisiere obisnuite:

- Blocarea la citire (blocking read) → Daca pipe-ul este gol, functia read blocheaza executia procesului si asteapta pana cand cel putin un octet este scris in pipe de un alt proces.
- Blocarea la scriere (blocking write) → Daca buffer-ul pipe-ului este plin (capacitatea standard este, de obicei, de 64 KB in Linux), apelul write blocheaza procesul pana cand se elibereaza spatiu de citire.
- Semnalul de sfarsit (EOF): Daca toate capetele de scriere ale unui pipe sunt inchise, o operatiune de read va returna 0, indicand faptul ca fluxul de date s-a terminat. Acesta e mecanismul prin care buclele de citire stiu sa se opreasca.
- Comportament atomic: Scrierea unor dimensiuni mici de date (mai mici decat PIPE_BUF) este atomica, adica sistemul de operare garanteaza ca datele nu se vor amesteca, char daca mai multe procese scriu simultan.

Bi-directional communication: use one pipe per direction

Pentru a realiza o comunicare bidirectionala intre doua procese (cum ar fi Procesul A si Procesul B sau Procesul A si Parintele) este necesar sa se utilizeze doua pipe-uri separate, cate unul pentru fiecare directie.

De ce nu putem folosi un singur pipe pentru ambele directii?

- Amestecarea datelor (Race Conditions): Daca ambele procese scriu si citesc din aceeasi conducta, datele s-ar suprapune. Un proces ar putea citi propriul mesaj in loc de cel trimis de partener.
- Problema citirii (Deadlock): Daca ambele procese incearca sa citeasca dintr-un singur pipe, iar acesta este gol, ambele se vor bloca, asteptand ca celalalt sa scrie (blocaj total/deadlock).

- Gestionarea corecta a EOF: Un singur pipe are doua capete. Daca ai un cititor si un scriitor, fluxul este unidirectional. Cu doua pipe-uri, fiecare proces stie exact care este capatul de intrare si cel de iesire.

Structura corecta pe cele doua canale:

Pipe 1: Procesul A scrie, Procesul B citeste (Directia A → B)

Pipe 2: Procesul B scrie, Procesul A citeste (Directia B → A)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc, char** argv) {
    int p2c[2], c2p[2];
    if (pipe(p2c) == -1 || pipe(c2p) == -1) {
        perror("Eroare la crearea pipe-urilor");
        exit(EXIT_FAILURE);
    }
    pid_t pid = fork();
    if (pid < 0) {
        perror("Eroare la fork");
        exit(EXIT_FAILURE);
    }
    //p2c[0] pentru citire
    //c2p[1] pentru scriere
    if (pid == 0) {
        close(p2c[1]); //nu trebuie sa scriem in p2c, nu scriem in pipe-ul Parinte -> Copil
        close(c2p[0]); //nu trebuie sa citim din c2p, nu citim din pipe-ul Copil -> Parinte
        int n;
        //citim valoarea trimisa de parinte
        if (read(p2c[0], &n, sizeof(int)) > 0) {
            printf("[Copil] Am primit de la parinte numarul: %d\n", n);
            n = n * 2;
```

```
            write(c2p[1], &n, sizeof(int));
            printf("[Copil] Am trimis inapoi catre parinte: %d\n", n);
        }
        close(p2c[0]);
        close(c2p[1]);
        exit(EXIT_SUCCESS);
    }
  //procesul parinte
    close(p2c[0]); //nu citim din Parinte -> Copil
    close(c2p[1]); //nu scriem in Copil -> Parinte
    int valoare_trimisa = 10;
    printf("[Parinte] Am trimis catre copil: %d\n", valoare_trimisa);
    write(p2c[1], &valoare_trimisa, sizeof(int));
    int rezultat;
    if (read(c2p[0], &rezultat, sizeof(int)) > 0) {
        printf("[Parinte] Am primit de la copil rezultatul: %d\n", rezultat);
    }
    close(p2c[1]);
    close(c2p[0]);
    wait(NULL);
    exit(EXIT_SUCCESS);
}
```

Separarea clara a canalelor: Parintele foloseste p2c[1] pentru a scrie si c2p[0] pentru a citi. Copilul foloseste c2p[1] pentru a scrie si p2c[0] pentru a citi.

Inchiderea descriptorilor: Imediat dupa fork(), fiecare proces inchide acele capete de pipe care nu sunt de folos in directia respectiva. Acest lucru previne blocajele (deadlock-urile) si permite functiilor read sa detecteze corect sfarsitul comunicarii (EOF).

Unix Fifo Communication

What if we want to communicate between processes that do not have a common ancestor written by us? Since pipes are only transmited through inheritance, we need another mechanism.

FIFOs are very similar in functionality to pipes and can be used to communicate between any processes.

The differences of using FIFO versus pipes are:

a. FIFOs are files on disk, and consequently they have a unique system-wide ID (the file path) that can be used by processes to addres them).
b. Pipe creation also opens them, but FIFOs need to be created and opened explicitly.
c. Pipes are destroyed when all their ends closed, FIFOs need to be deleted explicitly.
d. FIFOs may live beyond the processes that use them, and if not properly handled, may also keep data during executions, thus potentially creating problems.

How cand we create FIFOs?

a. Using the command mkfifo
   i. Example: mkfifo myfifo
b. Using the C instruction int mkfifo(const char *pathname, mode_t mode)
   i. Example: mkfifo(''myfifo", 0600)

Fifo acces control

a. Having an unexpected process read or write from your FIFO, will result in data being removed or added to the FIFO in ways your implementation does not expect
b. For the beginning, it is a good policy to create your FIFOs somewhere in your home directory, and give permissions only to yourself.

Pe scurt → having an unexpected process read/write from a FIFO → data being removed or added unexpectedly

Create your FIFO in your home directory, give permissions only to yourself

FIFO removal can be done either with the rm command on the int unlink(const char *path)

```
int main(int argc, char** argv) {

    int a2b, b2a, n;

    a2b = open("a2b", O_WRONLY); //se deschide A -> B pt scriere

    b2a = open("b2a", O_RDONLY); //se deschide B -> A pt citire

    n = 20;

    write(a2b, &n, sizeof(int)); //se trimite n

                    //scop: initializarea comunicarii (Bootstrap)

                    //procesul A deschide a2b pentru scriere si se blocheaza pana cand B deschide a2b pentru citire

                    //procesul B incearca sa citeasca din a2b (read(a2b, &n, sizeof(int)), dar se blocheaza pentru ca nu exista date in conducta

                    //pentru ca B sa inceapa sa lucreze, procesul A trebuie neaparat sa scrie cel putin o data un numar in a2b. Acesta este primul write(a2b, &n, sizeof(int)) -> n = 20
```

```
                        //fara aceasta trimitere initiala, ambele procese s-ar bloca in astesptarea datelor de la celalalt (deadlock)
        while(1) {
                if (read(b2a, &n, sizeof(int)) <= 0) {
                        break;
                }
                if (n <= 0) {
                        break;
                }
                printf("A: %d -> %d\n", n, n-1);
                n--;
                write(a2b, &n, sizeof(int)); //se trimite n
        }
        close(a2b);
        close(b2a);
        return 0;
}

int main(int argc, char** argv) {
        int a2b, b2a, n;
        a2b = open("a2b", O_RDONLY); //se deschide a2b pentru citire
        b2a = open("b2a", O_WRONLY); //se deschide b2a pentru scriere
        while(1) {
                if (read(a2b, &n, sizeof(int)) <= 0) {
                        break;
                }
                if (n <= 0) {
                        break;
                }
                printf("B: %d -> %d\n", n, n-1);
```

```
            n--;
            write(b2a, &n, sizeof(int));
        }
        close(a2b);
        close(b2a);
        return 0;
}
alexandra@LAPTOP-ALLE:~/processes$ ./prog16
B: 20 -> 19
B: 18 -> 17
B: 16 -> 15
B: 14 -> 13
B: 12 -> 11
B: 10 -> 9
B: 8 -> 7
B: 6 -> 5
B: 4 -> 3
B: 2 -> 1
alexandra@LAPTOP-ALLE:~/processes$ ./prog15
A: 19 -> 18
A: 17 -> 16
A: 15 -> 14
A: 13 -> 12
A: 11 -> 10
A: 9 -> 8
A: 7 -> 6
A: 5 -> 4
A: 3 -> 2
A: 1 -> 0
```

Special open behaviour for FIFO: waits for the FIFO to be open for the complementary operation before returning .

This is a very useful synchronization mechanism, as open ensures that a FIFO is open, there is another process ready to deal with this process operations (i.e. read what this process writes, or write so that this process can read.

Can lead to deadlock, if the FIFOs are not open in the same order. To try it, swap the open lines in program B and run the programs again.

Ways to communicate through pipe/FIFO

1. BYTE BUFFERS, prefixed by length

```
// WRITER int main(int argc, char**argv) {
int f, n;
char* s = "Hello!";
f = open("w2r", O_WRONLY);
n = strlen(s)+1;
write(f, &n, sizeof(int));
write(f, s, n);
close(f);
return 0;
}
// READER int main(int argc, char**argv) {
int f, n;
char* s;
f = open("w2r", O_RDONLY);
read(f, &n, sizeof(int));
s = (char*)malloc(n);
read(f, s, n);
free(s);
close(f);
return 0;
}
```

If sending strings in this way, it is a good idea to send the ending zero.

2. BYTE BUFFERS

Popen

- For running a program or any general shell command from C code, and getting back its standard output, or sending back its standard input, you need popen.

Popen() → este ca un ,,pod" intre programul tau si linia de comanda

In loc sa folosesc system() care executa o comanda si te lasa sa astepti fara sa poti intercepta ce afiseaza, popen() iti da acces direct la fluxul de date (input sau output) al acelei comenzi sub forma unui simplu pinter catre un fisier (FILE *).

FILE *popen(const char *command, const char *type);

Command → este comanda de shell pe care vrei sa o rulezi (exemplu: ls -la, grep text sau less)

Type poate fi: „r" → read → rulezi comanda si citesti ce afiseaza ea (output-ul ei devine input pentru codul tau)

,,w" → write → rulezi comanda si ii trimiti date (ce scrii tu in fisier devine input pentru comanda)

FILE *pager = popen("less", "w")

La final, fluxul trebuie inchis mereu cu pclose(pager) in loc de clasicul fclose()

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char** argv) {
    FILE* fp;
    fp = popen("less", "w");
    for (int i = 99; i>0; i--) {
        fprintf(fp, "%d bottles of beer on the wall,\n",i);
        fprintf(fp, "     %d bottles of beer.\n", i);
        fprintf(fp, "If one of those bottles should happen to fall,\n");
        fprintf(fp, "     %d bottles of beer in the wall.\n\n", i-1);
    }
    pclose(fp);
    return 0;
}
```

```
#include <stdio.h>

#include <stdlib.h>

int main(int argc, char** argv) {

    int n;

    char u[64];

    FILE* fp;

    fp = popen("ps -ef | awk '{print $1}' | sort | uniq -c | sort -nr | head -n 1", "r");

    fscanf(fp, "%d %s", &n, u);

    pclose(fp);

    printf("%s: %d\n", u, n);

    return 0;

}
```

Unix IPC Shared Memory

UNIX IPC is a set of mechanisms (other than pipe and FIFO) for communicating among processes (IPC = Inter-Process Communication)

a. Semaphore: synchronization mechanism
b. Message queues: message-based communication
c. Shared memory: communication through a common memory region

Spre deosebire de pipes (tevi) sau FIFOs (tevi cu nume) – care functioneaza ca nuste fluxuri continue de octeti (byte-streams) si au o structura destul de rigida de tip parinte-copil sau producator-consumator – aceste trei mecanisme ofera mult mai multa flexibilitate si control pentru aplicatiile complexe.

A . Semaphore : Mecanismul de sincronizare

Semafoarele nu sunt gandite pentru a transfera date (nu trimiti text sau numere prin ele), ci functioneaza ca niste semafoare de trafic sau contoare inteligente.

- Problema pe care o rezolva: Previn race-conditions → cand doua procese incearca sa modifice aceeasi sursa (un fisier, o baza de date) in acelasi timp.

B. Message Queues (Cozi de mesaje): Comunicare structurata

Daca trimiti prin pipes un text brut si trebuie sa inventezi un mod de a stii unde se termina un mesaj si unde incepe urmatorul, cozile de mesaje trimit pachete bine definite (structuri).

Avantajul major: Sunt asincrone. Asta inseamna ca Procesul A poate pune 10 mesaje in coada si poate sa isi inchida activitatea. Procesul B poate veni dupa 2 ore sa citeasca acele mesaje. Mesajele supravietuiesc in nucleul sistemului de operare (OS) chiar daca procesul care le-a creat a murit.

Bonus: Fiecare mesaj are un ”tip” (un numar de prioritate). Procesul B poate sa citeasca doar mesajele de tipul 3 (de exemplu, erori critice), ignorand momentan restul cozii.

C. Shared Memory (Memorie Partajata): Viteza suprema

Acesta este cel mai rapid mecanism de IPC disponibil in UNIX.

De ce este cel mai rapid? In mod noram, cand procesul A vrea sa vorbeasca cu procesul B prin pipe sau mesaje, datele sunt copiate din memoria procesului A in memoria nucleului (Kernel), si apoi din Kernel in procesul B. Asta inseamna timp pierdut.

Cum functioneaza: Shared memory cere sistemului de operare sa decupeze o bucata din memoria RAM si sa o mapreze (sa o lege) direct in spatiul ambelor procese. Nu exista intermediari.

Prapastia Shared Memory: Pentru ca shared memory este atat de rapida si procesele scriu direct acolo fara controlul sistemului de operare, ai nevoie obligatoriu de un Semafor (punctul A) care sa le coordoneze. Altfel, procesele vor scrie unul peste altul si vor corupe datele. De aceea, in UNIX, Shared Memory si Semafoarele merg aproape mereu mana in mana.

We will only look into shared memory

IPC identification

a. Just like FIFOs, IPCs allow any two processes to communicate through them, but unlike FIFOs, IPCs are not files on disk.
b. Identified by a number that is unique in the system
c. This number is chosen by the developer (explicitly or using specialized functions) and provided to all processes that need to communicate through that specific IPC.

Pentru ca nu sunt fisiere pe disc (cum sunt FIFOs), ele nu au o cale de tipul /tmp/my_fifo. In schimb, au nevoie de o ”adresa” abstracta. Aici intervin notiunile de Cheie (Key) si Identificator (ID).

Cum alegem acest numar? (Cheia IPC)

- Acest numar este stabilit de programator si trebuie sa fie cunoscut de toate procesele care vor sa vorbeasca intre ele. In UNIX, acest numar initial se numeste Cheie (ipc_key).

Exista doua metode prin care procesele cad de acord asupra acestei chei:

- Varianta Hardcoded (Explicita): Alegi pur si simplu un numar la intamplare, de exemplu 1234. Daca procesul A si Procesul B folosesc numarul 1234 in cod, ele vor accesa aceeasi resursa.
- Varianta dinamica (ftok): Pentru a evita situatia in care alt program de pe calculator foloseste deja numarul 1234, UNIX ofera functia ftok() (File to Key). Acesta genereaza o cheie unica pe baza unui fisier existent pe disc si a unui caracter ales de tine.

```
key_t my_key = ftok(”/etc/passwd”, ’R’);
```

Atata timp cat ambele procese apeleaza ftok cu exact aceeasi cale si acelasi caracter, ele vor obtine aceeasi cheie.

key_t my_key = ftok(”/etc/passwd”, ’R’);

Ce face functia ftok?

Nu se uita la continutul fisierului (nu citeste textul din el). Se uita la proprietatile acelui fisier din sistemul de fisiere UNIX. Extrage doua informatii unice pe care sistemul de operare le atribuie oricarui fisier de pe disc:

Numarul Inod-ului → identificator numeric unic al fisierului de pe acea partitie

ID-ul Dispozitivului (Device ID) → identificatorul partitiei/had-disk-ului pe care se afla fisierul

Sistemul de operare ia aceste doua numere, adauga un caracter ales de tine, si le impacheteaza pe toate trei intr-un singur numar de tip int, numit key_t.

Cum identifica sistemul resursa? (ID-ul IPC)

Odata ce ai cheia, o trimiti sistemului de operare ca sa creezi sau sa deschizi resura (folosing functii precum shmget pentru shared mempry, msgget pentru cozi de mesaje, sau semget pentru semafoare).

Sistemul de operare preia cheia ta si iti returneaza un identificator unic in sistem (ID) care este exact numarul mentionat.

// Procesul A cere o coadă de mesaje folosind cheia stabilită

int msg_id = msgget(my_key, IPC_CREAT | 0666);

Din acest moment, in interiorul programului tau nu mai folosesti cheia, ci folosesti acest msg_id pentru a trimite sau primi date.

Cum le poti vedea in actiune chiar acum> IPD traiesc direct in memoria RAM gestionata de kernel si nu lasa urma pe disc => UNIX pune la dispozitie doua comenzi de terminal extrem de utile pentru a le spiona sau curata:

1. ipcs (IPC Status): Afiseaza instant toate cozile de mesaje, zonele de memorie partajata si semafoarele active in calculator, impreuna cu cheia si ID-ul lor.
2. ipcrm (IPC Remove): Daca un program s-a blocat si a lasat o zona de memorie ocupata de pomana, o poti sterge manual din terminal folosind ID-ul ei (ipcrm -m <ID_MEMORIE>).

IPC cleanup

a. IPCs are persistent structures in the operating system
b. The operating system imposes limits for the size and number of IPCs
c. If not cleanup properly, the system will refuse to allow creation of new IPCs
d. You can see all existing IPCs using the command ipcs
e. You can delete an IPC using command ipcrm, as long as you have permissions on that IPC

Shared memory API

a. Create or get a handle to an existing shared memory segment: shmget
b. Attach – map the shared memory segment to a pointer in your process: shmat
c. Detach – unmap the shared memory segment from your pointer: shmdt
d. Control (configure, delete, etc.) the shared memory segment: shmctl

Header.h

```
struct absp {
    int a; //primul numar scris de programul A
    int b; //al doilea numar scris de programul A
    int s; //suma calculata si scrisa de programul B
    int p; //produsul calculat si scris de programul B
};
```

Program A

```
#include <sys/shm.h>
#include <stdio.h>
#include "header.h"
int main() {
    int shmid, k = 0;//shmid va tine ID-ul memoriei, iar 'k' este un contor pentru numere
    struct absp *x;//un pointer catre structura din memoria partajata

    shmid = shmget(1234, sizeof(struct absp), IPC_CREAT | 0600);
    //cere sistemului de operare o zona de memorie partajata cu cheia fixa        //1234
    //Dimensiunea ceruta este exact dimensiunea structurii noastre
    //(16 octeti)
    //Flag-ul IPC_CREAT spune "Creeaz-o daca nu exista", iar 0600 inseamna        //ca doar utilizatorul care a pornit acest program are drept de citire        //si scriere in ea
    x = shmat(shmid, 0, 0);
    //Conecteaza (attach) segmentul de memorie cu ID-ul shmid in spatiul
    //de adrese al programului A
    //x puncteaza direct in memoria RAM globala
    while(1) {
        x->a = k++ % 100;//pune in memoria partajata valoarea lui k,
```

```
                    //apoi o incrementeaza
            x->b = k++ % 100;//face acelasi lucru pt b
            if (x->p == x->s && x->p != 0) {
                  break;
            }//daca in memoria partajata produsul (p) devine egal cu suma
             //(s), bucla se opreste
      }

      shmdt(x);//deconecteaza memoria partajata de pointerul x
                  //Programul A nu mai are acces la ea
      shmctl(shmid, IPC_RMID, NULL);//distruge definitiv segmentul
                              //de memorie din tot sistemul de                              //operare
      printf("Programele s-au sincronizat si memoria a fost curatata!\n");
      return 0;
}Program B
#include <stdio.h>
#include <sys/shm.h>
#include "header.h"
int main() {
      int shmid;
      struct absp *x;
      shmid = shmget(1234, 0, 0);//cauta in segmentul de memorie partajata
                        //cu cheia 1234
      x = shmat(shmid, 0, 0);//conecteaza acea memorie la pointer-ul
                        //local x
                        //acum pointer-ul x din programul B se uita la                        //exact aceeasi
bucata de RAM ca si pointer-ul                        //x din programul A
      while (1) {
            x->s = x->a + x->b;//citeste a si b din memorie, le aduna si
                        //scrie rezultatul in s
```

```
            x->p = x->a * x->b;//citeste a si b din memorie, le inmulteste                    //si scrie
rezultatul in p
            if (x->p == x->s) {
                  break;//daca produsul calculat este egal cu suma
                      //calculata, opreste bucla
            }
      }
      shmdt(x);//deconecteaza memoria partajata de programul B
            //programul B nu mai are linia shmctl(..., IPC_RMID, ...)
            //responsabilitatea stergerii a ramas la programul A
      return 0;
}
```

// scriitor.c

```
#include <stdio.h>
#include <sys/shm.h>
#include <unistd.h>

int main() {
   // 1. Creează o memorie partajată cu cheia 9999, destul de mare pentru un singur INT
   int shmid = shmget(9999, sizeof(int), IPC_CREAT | 0666);

   // 2. Conectează memoria la un pointer de tip int
   int *numar = (int *) shmat(shmid, NULL, 0);

   // 3. Pune valoarea 42 în RAM-ul partajat
   *numar = 42;
   printf("[Programul A] Am scris valoarea %d in memoria partajata.\n", *numar);
   printf("[Programul A] Astept 10 secunde... Porneste repede Programul B!\n");

   // Îi dăm timp programului B să citească valoarea înainte să ștergem memoria
```

```
    sleep(10);

    // 4. Curățenie
    shmdt(numar);
    shmctl(shmid, IPC_RMID, NULL);
    printf("[Programul A] Memoria a fost stearsa. Jocul s-a terminat.\n");

    return 0;
}
// cititor.c
#include <stdio.h>
#include <sys/shm.h>

int main() {
    // 1. Caută memoria cu cheia 9999
    int shmid = shmget(9999, 0, 0);
    if (shmid == -1) {
        printf("[Programul B] Eroare: Memoria nu exista! Porneste mai intai Programul A.\n");
        return 1;
    }

    // 2. Conectează memoria la pointerul local
    int *numar = (int *) shmat(shmid, NULL, 0);

    // 3. Citește și printează valoarea direct din RAM
    printf("[Programul B] Am citit din memoria partajata valoarea: %d\n", *numar);

    // 4. Se deconectează
    shmdt(numar);
```

```
    return 0;
}
```

a. The programs above will not function in any way close to what we want, because we have no mechenisms to ensure they work on the shared memory in turns. Instead, they work ignoring the presence of the other.
b. If the programs are started manually one by one, it is likely that program A deletes the shared memory before B is done with it, resulting in errors. This can happen if A finds p and s being equal simply because their value happens to be zero, and then breaks from the loop and deletes the shared memory.
c. So, shared memory without synchronization mechanisms is guaranteed to cause trouble.

POSIX THREADS

Pthread creation and sheduling

Threads are another mechanism for implementing concurrent programs that allow faster creation, reduced memory usage, and much simpler communication. POSIX threads (Pthreads) are the native threads implementation in Linux, but every modern operating system provides thread creation libraries.

Bellow is a very basic thread creation example, using the Pthreads library.

```
#include <stdio.h>

#include <pthread.h>

void* f(void* a) {//void* inseamna ca functia returneaza un pointer generic (adica o adresa de memorie catre orice tip de date)

    printf("f\n");//void* a reprezinta argumentul pe care functia il primeste. Chiar daca nu il folosim aici, structura este obligatorie pentru pthread

    return NULL;

}

int main(int argc, char** argv) {

    pthread_t t;//declaram variabila t de tipul pthread

    pthread_create(&t, NULL, f, NULL);//da nastere noului fir de executie

    //NULL atributele thread-urilor, setari implicite de securitate/memorie

    //f = numele functiei pe care noul thread trebuie sa o execute imediat        //ce porneste

    //argumentul pe care vrem sa il trimitem functiei f, in cazul nostru, nimic

    printf("main\n");//in timp ce thread-ul proaspat creat incearca sa porneasca si sa ruleze functia f, procesul principal isi vede de drum si afiseaza main

    pthread_join(t, NULL);//bariera de siguranta
```

```
                //pthread_join ii spune programului: opreste-te aici pana cand thread-ul t isi termina complet treaba

                //daca nu am pune aceasta linie, programul principal s-ar putea inchide atat de repede incat thread-ul t nici n-ar apuca sa afiseze litera "f"

    return 0;

}
```

A thread always executes a given function, in our case f().

```
    pthread_create(&t, NULL, f, NULL)
```

The Pthread library requires the thread function to have a specific signature. It should have a single void* argument and it should return void*.

The call to pthread_create creates and starts a thread that executes the given function pointer f, and populates the thread handler t.

The second argument to pthread_create is a pointer to pthread_attr_t which controls various aspects of the creation and execution.

The fourth argument to pthread_create is the argument to be passes to the function f(). In our case, f() doesn't use its argument, so we pass NULL.

```
pthread_join(t, NULL);
```

The call to pthread_join waits for the thread identified by the handle t to finish.

The second argument to pthread_join will contain the value returned by function f(). Passing it as NULL means we do not need this value, so we ignore it.

To compile a program that uses Pthreads, you need to either pass the -pthread argument to gcc, or tell it to use the Pthreads library, which you can do passing the -lpthread argument.

```
gcc -Wall -Wextra -Werror -g -o a a.c -pthread
```

or

```
gcc -Wall -Wextra -Werror -g -o a a.c -lpthread
```

The output of this program depends on how the operating system shedules the execution of these threads, consequently, we may see the two printf-ed values displayed in any order.

This is essential to understanding how concurrency works.

It is difficult to see the non-deterministic aspect of thread sheduling using the program above, because the thread execution is extremely short, and it takes a lot of trials to see the lines printed in a different order. Running the program on a busier computer would make this a little easier to see, but we can make this easily obvious using thread functions that do a lot more.

```
#include <stdio.h>

#include <pthread.h>
```

```
int n = 1;
void* fa(void* a) {
	for (int i = 0; i < n; i++) {
		printf("fa\n");
	}
	return NULL;
}
void* fb(void* a) {
	for (int i = 0; i < n; i++) {
		printf("fb\n");
	}
	return NULL;
}
int main(int argc, char** argv) {
	pthread_t ta, tb;//declaram variabile de tipul pthread
	if (argc > 1) {
		sscanf(argv[1], "%d", &n);//primim un numar n ca argument
	}
	pthread_create(&ta, NULL, fa, NULL);//da nastere noului fir de executie
	pthread_create(&tb, NULL, fb, NULL);//da nastere noului fir de executie
	pthread_join(ta, NULL);
	pthread_join(tb, NULL);
	return 0;
}
```

We now have two thread functions that is each run in a separate thread.

The command line argument specifies how many iterations each thread will do. Notice that each thread has acces to the global variables.

To show the thread scheduling without having long printouts, we pipe the output of the program through uniq -c without sorting the out first, thus displaying how many iterations each thread did before the scheduler switched to another. Bellow are two executions of the same program, taken a few moments apart.

alexandra@LAPTOP-ALLE:~/others$ ./prog4 10000 | uniq -c

166 fa

1 fb

1 fa

5 fb

1 fa

3 fb

1 fa

1 fb

2 fa

1 fb

5 fa

1 fb

6 fa

1 fb

7 fa

1 fb

6 fa

1 fb

4 fa

1 fb

3 fa

...

./prog4 10000 | uniq -c

- Am rulat programul (care are doua thread-uri, unul care scrie fa si unul care scrie fb, repetate de 10000 de ori)
- | (pipe) → trimite tot ce afiseaza programul tau catre urmatoarea comanda, in loc sa le printeze pe ecran
- uniq -c → Numara liniile identice care apar una dupa alta

Daca thread-urile ar fi rulate pe rand (intai tot thread-ul A, apoi tot thread-ul B), ai fi vazut doar doua linii mari 10000 fa si 10000 fb

Dar pentru ca procesorul le ruleaza in paralel, el le tot schimba intre ele foarte rapid (procesul se numeste context switching). Rezultatul arata exact cum a impartit procesorul timpul:

- 166 fa: La inceput, thread-ul fa a prins prioritate si a apucat sa printeze 166 de ori consecutiv inainte ca sistemul de operare sa il puna pe pauza
- 1 fb: Sistemul de operare a zis ”stop” lui fa si i-a dat voie lui fb. Dar fb a apucat sa scrie o singura data, pentru ca imediat i-a expirat timpul alocat sau a fost intrerupt.
- 1 fa: Procesorul a mutat executia inapoi la primul thread, care a scris o data
- 5 fb: A urmat iar thread-ul doi, care a avut un pic mai mult timp si a scris de 5 ori la rand

Ordinea si numarul de repetari de fiecare data vor fi mereu diferite, pentru ca depind de cat de ocupat este procesorul in acea milisecunda.

alexandra@LAPTOP-ALLE:~/others$ ./prog4 10000 | uniq -c

1965 fa

10 fb

1 fa

6 fb

14 fa

1 fb

1 fa

3 fb

1 fa

1 fb

4 fa

...

Acelasi program, cu printf main:

```
#include <stdio.h>
#include <pthread.h>
int n = 1;
void* fa(void* a) {
    (void)a;
    for (int i = 0; i < n; i++) {
        printf("fa\n");
    }
```

```
	return NULL;
}
void* fb(void* a) {
	(void)a;
	for (int i = 0; i < n; i++) {
		printf("fb\n");
	}
	return NULL;
}
int main(int argc, char** argv) {
	pthread_t ta, tb;//declaram variabile de tipul pthread
	if (argc > 1) {
		sscanf(argv[1], "%d", &n);//primim un numar n ca argument
	}
	pthread_create(&ta, NULL, fa, NULL);//da nastere noului fir de executie
	pthread_create(&tb, NULL, fb, NULL);//da nastere noului fir de executie
	for (int i = 0; i < n; i++) {
		printf("main\n");
	}
	pthread_join(ta, NULL);
	pthread_join(tb, NULL);
	return 0;
}
alexandra@LAPTOP-ALLE:~/others$ ./prog4 10000 | sort | uniq -c
  10000 fa
  10000 fb
  10000 main
alexandra@LAPTOP-ALLE:~/others$ ./prog4 10000 | uniq -c
    477 fa
      1 fb
```

```
 9 fa
16 fb
 1 fa
 1 fb
 4 fa
 1 fb
 3 fa
 8 fb
 1 fa
 2 fb
 1 fa
 1 fb
 1 fa
 1 fb
 5 fa
 5 fb
 1 fa
...
```

Pthread argument passing

The last program can be written using a single thread function that gets value to display in the console as argument

```
#include <stdio.h>
#include <pthread.h>
int n = 1;
void* f(void* a) {
	for (int i = 0; i < n; i++) {
		printf("%s\n", (char*)a);
	}
	return NULL;
}
```

```
int main(int argc, char** argv) {
	pthread_t ta, tb;
	if (argc > 1) {
		sscanf(argv[1], "%d", &n);
	}
	pthread_create(&ta, NULL, f, "fa");
	pthread_create(&tb, NULL, f, "fb");

	for (int i = 0; i < n; i++) {
		printf("main\n");
	}
	pthread_join(ta, NULL);
	pthread_join(tb, NULL);
	return 0;
}
```

```
alexandra@LAPTOP-ALLE:~/others$ ./prog5 1000 | uniq -c
    728 fa
      1 main
     21 fa
      1 main
      4 fa
      1 main
      6 fa
      1 main
      1 fa
      4 main
      1 fa
      3 main
alexandra@LAPTOP-ALLE:~/others$ ./prog5 1000 | sort | uniq -c
   1000 fa
```

1000 fb

1000 main

If we want to pass n as an argument to the thread too (instead of having it a global variable), we need to declare a struct since the thread function can only get one argument.

```
#include <stdio.h>

#include <pthread.h>

//Definim o structura personalizata pentru a putea grupa datele pe care vrem

//sa le trimitem fiecarui thread

struct arg_t {

	char* name;//un pointer catre un sir de caractere (textul pe care

		//il va afisa thread-ul

	int count;//un numar intreg care specifica de cate ori sa se repete

		//afisarea

};

//functia pe care o vor rula thread-urile

void* f(void* a) {

	//convertim point-erul generic void* a inapoi la tipul corect: un pointer catre structura noastra struct arg_t*

	struct arg_t* x = (struct arg_t*)a;


	for (int i = 0; i < x->count; i++) {

		printf("%s\n", x->name);//afisam pe ecran textul ('name') specific acestui thread, urmat de linie noua

	}

	return NULL;//functia isi termina executia si returneaza NULL (cerinta obligatorie pentru tipul void*)

}

int main(int argc, char** argv) {

	int n = 1;

	pthread_t ta, tb;//declaram identificatorii (ID-urile) pentru cele doua thread-uri pe care le vom crea

	struct arg_t aa, ab;//cream doua instante ale structurii noastre: 'aa' pentru thread-ul A si 'ab' pentru thread-ul B
```

```
    //verificam daca utilizatorul a trimis un argument in linia de comanda        //(ex: ./prog4 100000)
    if (argc > 1) {
        sscanf(argv[1], "%d", &n);
    }
   //initializam datele pentru primul thread (ce nume sa aiba si de cate ori sa ruleze)
    aa.name = "fa";
    aa.count = n;
    //initializam datele pentru al doilea thread
    ab.name = "fb";
    ab.count = n;
    //cream thread-ul "ta". Acesta va rula functia "f" si va primi ca argument adresa structurii "aa" (&aa)
    pthread_create(&ta, NULL, f, &aa);
    //cream thread-ul "tb". Acesta va rula functia "f", dar va primi adresa structurii "ab" (&ab)
    pthread_create(&tb, NULL, f, &ab);
                                                    for(int i = 0; i < n; i++) {
        printf("main\n");                                   }
    //firul principal se opreste aici si asteapta ca thread-ul "ta" sa isi termine complet executia
    pthread_join(ta, NULL);
    //firul principal asteapta ca si thread-ul "tb" sa isi termine treaba
    pthread_join(tb, NULL);
    return 0;//programul s-a incheiat cu succes si returneaza codul 0
         //catre sistemul de operare
}
```

Acest cod aduce o imbunatatire uriasa fata de versiunea anterioara: nu mai foloseste o variabila globala, foloseste o structura (struct) pentru a trimite data personalizate catre fiecare thread in parte. Prin urmare, ambele thread-uri folosesc acum aceeasi functie (f) , dar fiecare primeste argumente diferite.

```
alexandra@LAPTOP-ALLE:~/others$ ./prog6 1000 | uniq -c
    652 fa
      6 fb
     11 fa
      1 fb
```

5 fa

1 fb

1 fa

7 fb

1 fa

1 fb

2 fa

1 fb

1 fa

1 fb

9 fa

1 fb

...

Let’s create 10 threads, pass to each of them the order number in whitch it was created, and have it displayed. The program will include a serios bug that causes to a race condition and yields puzzling outputs.

```
#include <stdio.h>

#include <pthread.h>

void* f(void* a) {

	printf("%d\n", *((int*)a));//dereferentierea unui pointer

				//citeste valoarea de la acea adresa

	return NULL;

}

int main(int argc, char** argv) {

	(void)argc;

	(void)argv;

	pthread_t t[10];//declaram un tablou (array) de 10 identificatori de thread. Vom avea 10 fire de executie

	//crearea celor 10 thread-uri

	for (int i = 0; i < 10; i++) {

		//cream thread-ul cu numarul i
```

```
        //ii dam ca argument adresa lui i (&i), adica ii spunem
        //uite unde se afla numarul tau
        pthread_create(&t[i], NULL, f, &i);
    }
    //a doua bucla for: asteptarea thread-urilor
    for (int i = 0; i < 10; i++) {
        //firlu principal se asigura ca toate cele 10 thread-uri isi
        //termina treaba inainte ca programul sa se inchida
        pthread_join(t[i], NULL);
    }
    return 0;
}
```

```
alexandra@LAPTOP-ALLE:~/others$ ./prog8
1
3
4
3
5
6
7
8
9
10
```

→ Duplicate and missing numbers numbers

The root cause of this behaviour is the passing of &i to all threads. While we intend to pass to each thread the order number at which it was created, in reality, we pass to all threads the same value: the adress of i.

The main function changes the value of i in two for cycles (one for thread creation and one for joining).

Depending how the threads get the CPU, they will print whatever they find at the memory location &i.

To avoid this situation, we can either have an array of ints and send each thread the address of another element (first solution bellow), or dynamically allocate an int before creating each thread (second solution bellow).

Pentru a evita aceasta situatie, putem fie sa folosim un vector (array) de intregi si sa trimitem fiecarui thread adresa unui element diferit (prima solutie), fie sa alocam dinamic un intreg in memorie inainte de a crea fiecare thread (a doua solutie).

Solutia 1: Folosirea unui vector (a[10])

Prima solutie:

```
#include <stdio.h>

#include <pthread.h>

void* f(void* a) {
	printf("%d\n", *(int*)a);//dereferentierea unui pointer
	return NULL;
}

int main(int argc, char** argv) {
	(void)argc;
	(void)argv;
	int a[10];
	pthread_t t[10];
	for (int i = 0; i < 10; i++) {
		a[i] = i;
		pthread_create(&t[i], NULL, f, &a[i]);
	}
	for (int i=0; i < 10; i++) {
		pthread_join(t[i], NULL);
	}
	return 0;
}
```

A doua solutie

```
#include <stdio.h>

#include <stdlib.h>
```

```
#include <pthread.h>
void* f(void* a) {
	printf("%d\n", *(int*)a);
	free(a);//deoarece main-ul a alocat memorie special pentru acest
		//thread folosind malloc, este datoria thread-ului sa o
		//elibereze cand termina treaba
	return NULL;
}
int main(int argc, char** argv) {
	(void)argc;
	(void)argv;
	int* p;
	pthread_t t[10];
	for (int i = 0; i < 10; i++) {
		p = (int*)malloc(sizeof(int));
		*p = i;
		pthread_create(&t[i], NULL, f, p);//cream thread-ul si ii dam
//ca argument direct valoarea lui p
	}
	for (int i = 0; i < 10; i++) {
		pthread_join(t[i], NULL);
	}
	return 0;
}
```

Corectare: (A treia solutie)

```
// În main:
pthread_create(&t[i], NULL, f, (void*)(long)i); // Trimitem valoarea lui i direct ca pointer
// În funcția f:
int valoare = (int)(long)a; // Recuperăm valoarea direct, fără steluță (*)
printf("%d\n", valoare);
```

```
#include <stdio.h>
#include <pthread.h>
void* f(void* a) {
	int valoare = (int)(long)a;
	printf("%d\n", valoare);//dereferentierea unui pointer
				//citeste valoarea de la acea adresa
	return NULL;
}
int main(int argc, char** argv) {
	(void)argc;
	(void)argv;
	pthread_t t[10];//declaram un tablou (array) de 10 identificatori de thread. Vom avea 10 fire de executie
	//crearea celor 10 thread-uri
	for (int i = 0; i < 10; i++) {
		//cream thread-ul cu numarul i
		//ii dam ca argument adresa lui i (&i), adica ii spunem
		//uite unde se afla numarul tau
		pthread_create(&t[i], NULL, f, (void*)(long)i);
	}
	//a doua bucla for: asteptarea thread-urilor
	for (int i = 0; i < 10; i++) {
		//firlu principal se asigura ca toate cele 10 thread-uri isi
		//termina treaba inainte ca programul sa se inchida
		pthread_join(t[i], NULL);
	}
	return 0;
}
```

- Notice where we free the allocated memory in the second solution. What would happen if we freed it right after the call to pthread_create?

For-ul ar arata asa:

```
for(i=0; i<10; i++) {
    p = (int*)malloc(sizeof(int));
    *p = i;
    pthread_create(&t[i], NULL, f, p);
    free(p); // ❌ CE SE ÎNTÂMPLĂ DACĂ PUNEM FREE AICI?
}
```

Ce s-ar intampla?

Cursa contra-cronometru (Race Condition)

Functia pthread_create doar initializeaza thread-ul si ii spune sistemului de operare sa porneasca thread-ul, insa pornirea thread-ului dureaza cateva microsecunde.

In acest timp, firul principal (main) nu sta pe loc, executa imediat linia urmatoare: free(p). Memoria abia alocata este stearsa inainte ca thread-ul nou creat sa porneasca sau sa execute linia printf.

⇨ Undefined behaviour

Cand thread-ul se trezeste si incearca sa execute linia:

printf(”%d\n”, *(int*)a));

El va incerca sa citeasca dintr-o adresa de memorie (a) care nu ii mai apartine, pentru ca main-ul a eliberat-o deja.

In programare, acest lucru se numeste Use-After-Free, si poate duce la trei rezultate complet imprevizibile:

⇨ Gunoaie din memorie: Thread-ul va afisa numere complet ciudate si aleatorii
⇨ Crash direct (Segmentation Fault): Sistemul de operare observa ca thread-ul incearca sa acceseze ilegal o zona de memorie eliberata si opreste fortat tot programul
⇨ Iluzia ca functioneaza: Datele nu au fost inca sterse fizic din RAM de catre sistem si programul

Regula de aur in Pthreads:

Cel care foloseste ultimul memoria este cel care trebuie sa o elibereze. De acees, in solutia corecta, free(a) se afla la sfarsitul functiei f. Astfel, ne asiguram ca memoria a fost distrusa numai dupa ce thread-ul a terminat de printat valoarea respectiva.

The outputs of these implementations will be non-deterministic due to the order of the printfs, but they will contain the expected values.

POSIX SYNCHRONIZATION MECHANISMS

A program that creates 10 threads, each of whitch increments a global variable as many times given as a command line argument:

```
#include <stdio.h>
#include <pthread.h>
```

```
int count = 0;
void* f(void* a) {
	for (int i = 0; i < *(int*)a; i++) {
		count++;
	}
	return NULL;
}
int main(int argc, char** argv) {
	int n = 10;
	pthread_t t[10];
	if (argc > 1) {
		sscanf(argv[1], "%d", &n);
	}
	for (int i = 0; i < 10; i++) {
		pthread_create(&t[i], NULL, f, &n);
	}
	for (int i = 0; i < 10; i++) {
		pthread_join(t[i], NULL);
	}
	printf("%d\n", count);
	return 0;
}
```

Acest cod introduce o noua problema extrem de faimoasa din programarea paralela: Race Condition pe o variabila globala partajata (o cursa critica).

Pentru valori mari => Rezultate imprevizibile

alexandra@LAPTOP-ALLE:~/threads$ ./prog2 10000

100000

alexandra@LAPTOP-ALLE:~/threads$ ./prog2 100000

198678

alexandra@LAPTOP-ALLE:~/threads$ ./prog2 1000000

1559971

Problema este ca operatia count++ pare o singura instructiune C, dar in realitate procesorul (la nivel de cod masina/asamblare) o executa in 3 pasi distincti:

1. Read (Citire): Ia valoarea curenta a lui count din RAM si o aduce in interiorul procesorului

(intr-un registru)

2. Modify: Adauga 1 la acea valoare
3. Write (Criere): Pune valoare inapoi in memoria RAM, la adresa lui count

Pentru ca thread-urile ruleaza complet in paralel pe nuclee diferite ale procesorului, pasii lor se amesteca. Imagineaza-ti ca count este in prezent 50. Doua thread-uri (Thread A si Thread B) vor sa faca count++ in aceeasi microsecunda:

Cand ai 10 thread-uri care fac asta de 100000 ori, mii de incrementari se pierd din cauza acestor suprapuneri.

Pentru a repara asta, avem nevoie de un mecanism numit Mutex (Mutual Exclusion) – un fel de ”lacat” care blocheaza accesul celorlalte thread-uri in timp ce unul singur face operatia count++.

(ASM): lock inc dword ptr [count]

This situation is called race codition, with count being the critical resource and the line count++ being the critical section

Race conditions are solved using synchronization mechanisms.

MUTEXES

A mutex is a synchronization mechanism that has two main operations: lock and unlock.

Only one thread can complete the lock operation at one time, any other threads attempting to lock the mutex will have to wait until the ”winning” thread calls unlock.

The race condition in the code above can be avoided usig a mutex as shown bellow.

In libraria Pthreads din C, aceste doua operatii se traduc prin doua functii foarte simple:

➔ pthread_mutex_lock(&m): Inainte de a atinge variabila globala, thread-ul incearca sa incuie lacatul
➔ pthread_mutex_unlock(&m): Dupa ce a terminat treaba cu variabila globala, thread-ul descuie lacatul ca sa poata intra si altii.

Daca ai 10 thread-uri care ajung in acelasi timp la linia pthread_mutex_lock:

- Unul singur va fi castigatorul: Sistemul de operare ii permite doar unui sigur thread sa execute cu succes operatia de blocare
- Ceilalti asteapta la coada: Celelalte 9 thread-uri sunt puse instantaneu pe pauza (intra in starea suspendat).

- Eliberarea: Cand thread-ul castigator termina de executat instructiunile acelea in asamblare (mov, add, mov) si apeleaza pthread_mutex_unlock, sistemul de operare trezeste urmatorul thread din coada si ii da lui cheia.

```
pthread_mutex_t m; // Declaram un mutex global (lacătul nostru)
void* f(void* a) {
    int i;
    for(i=0; i<*(int*)a; i++) {
        pthread_mutex_lock(&m);   // 🔒 Încuie ușa! Nimeni altcineva nu mai poate intra aici.
        count++;                  // 🏎️ Zonă Critică: Acum instrucțiunile de asamblare
                                  // (mov, add, mov) sunt complet sigure!
        pthread_mutex_unlock(&m); // 🔓 Descuie ușa! Următorul thread la rând poate intra.
    }
    return NULL;
}
```

Solutie:

```
#include <stdio.h>
#include <pthread.h>
int count = 0;
pthread_mutex_t m;//Declaram variabila globala "m" de tip mutex
                  //acesta esre "lacatul" nostru
void* f(void* a) {
        for (int i = 0; i < *(int*)a; i++) {
                pthread_mutex_lock(&m);//thread-ul incearca sa incuie lacatul
//daca deja e incuiat de alt thread, acesta se opreste si asteapta la coada
                count++;
                pthread_mutex_unlock(&m);//thread-ul descuie lacatul, oferind
//sansa urmatorului thread din coada sa intre
        }
        return NULL;//thread-ul isi termina executia si returneaza NULL
}
```

```
int main(int argc, char** argv) {
	int n = 10;
	pthread_t t[10];//un vertor in care vom salva ID-urile celor 10
			//thread-uri pe care le cream
	if (argc > 1) {
		sscanf(argv[1], "%d", &n);
	}
	pthread_mutex_init(&m, NULL);//initializam mutex-ul "m" inainte de a
	//crea thread-urile
	//al doilea parametru (NULL) aplica setarile standard (implicite) ale
	//sistemului
	for (int i = 0; i < 10; i++) {
		//cream thread-ul cu numarul "i", ii spunem sa ruleze functia		//"f" si ii dam adresa lui "n" (&n) ca argument
		pthread_create(&t[i], NULL, f, &n);
	}
	for (int i = 0; i < 10; i++) {
		//firul principal se opreste si asteapta ca thread-ul "t[i]"
		//sa isi termine complet treaba
		pthread_join(t[i], NULL);
	}
	pthread_mutex_destroy(&m);//distrugem mutex-ul "m"
	//este o regula de igiena digitala: eliberam resursele folosite de
	//sitemul de operare pentru gestionarea acestui lacat, deoarece
	//programul se termina
	printf("%d\n", count);//afisam valoarea finala a lui "count". Acum va
			//fi mereu 10*n
	return 0;
}
```

What if we move the lock and unlock operations outside the for loop?

```
void* f(void* a) {
    pthread_mutex_lock(&m); // 🔒 MUTAT ÎNAINTE DE FOR
    for (int i = 0; i < *(int*)a; i++) {
        count++;
    }
    pthread_mutex_unlock(&m); // 🔓 MUTAT DUPĂ FOR
    return NULL;
}
```

1. Programul devine secvential: Primul thread care apuca sa execute pthread_mutex_lock va incuia lacatul si va intra in bucla for. El va face toate cele n incrementari (de exemplu, 100000 de adunari) singur-singurel.

In tot acest timp, celelalte 9 thread-uri sunt complet blocate la linia de start, asteptand ca primul thread sa termine intreaga bucla si sa execute pthread_mutex_unlock. Abia atunci va intra al doilea thread, va face si el toate cele 100000 de adunari de unul singur, si tot asa.

2. De ce este o problema? Desi rezultatul final va fi in continuare corect (1000000), ai pierdut complet avantajul de a avea un procesor cu mai multe nuclee.

Programul tau va rula exact ca si cum ai fi avut un singur fir de executie vechi si clasic.

Consumi memorie si resurse ca sa creezi 10 thread-uri, dar le obligi sa ruleze pe rand, unul dupa altul

Marea dilema a programarii? – trebuie sa gasim mereu un compromis

Strategia: Lock in interiorul buclei

Avantaje: Concurenta maxima: thread-urile ruleaza in paralel bucla for. Doar cand ajung la count++ se sincronizeaza pentru o nanosecunda

Dezavantaje: Overhead mare: functiile lock si unlock sunt apelate de milioane de ori, ceea ce consuma timp

Strategia: Lock in afara buclei

Avantaje: Fara overhead. Functiile lock si unlock sunt apelate doar de 10 ori in total (o singura data per thread)

Dezavantaje: Concurenta zero. Thread-urile nu mai ruleaza deloc in paralel; codul devine pur secvential

The second argument to pthread_mutex_init is a pointer to pthread_mutexattr_t which controls various aspects of the mutex creation and execution. In our case, by setting it to NULL, we use the default settings.

pthread_mutex_init(&m, NULL);

Thread-uri (Curs youtube)

Curs 8

Cream 10 procese si le tiparim:

Cu fork():

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
int main() {
	for (int i = 0; i < 10; i++) {
		if (fork() == 0) {
			printf("%d\n", getpid());
			exit(0);
		}
	}
	for (int i = 0; i < 10; i++) {
		wait(0);
	}
	return 0;
}
```

```
alexandra@LAPTOP-ALLE:~/threads$ ./prog4
3391
3392
3393
3394
3395
3396
```

3397

3398

3399

3400

Cu thread-uri:

```
#include <stdio.h>
#include <pthread.h>
void* f(void* a) {
	(void)a;
	printf("F: %ld\n", pthread_self());
	return NULL;
}
int main(int argc, char** argv) {
	(void)argc;
	(void)argv;
	pthread_t t[10];
	for (int i = 0; i < 10; i++) {
		pthread_create(&t[i], NULL, f, NULL);
	}
	for (int i = 0; i < 10; i++) {
		pthread_join(t[i], NULL);
	}
	return 0;
}
```

Cu thread-uri:

- Scriem o functie si ii spunem programului: ruleaza functia asta intr-un thread separat (cu fork() duplicam procesul)
- Thread-urile vor avea un handler => daca cream 10 thread-uri vom avea 10 handle-uri

Pthread_create(&t[i], NULL, f, NULL);

,,Creeaza un thread, pune-i handler-ui in acest obiect: &t[i], nu ii dam niciun atribut thread-ului, functia care trebuie sa fie executata in thread, NULL este argumentul acelei functii

f → pointer la functie

main va rula in thread-ul principal

f va rula intr-un thread al ei

Proces:

- Context (contine detalii id proces, permisiuni)
- Sursa (codul sursa)
- Constante (tot ce ii hardcodat)
- Variabile
- Heap → Locul unde se aloca memorie dinamica cu malloc
- Stack → Se inregistreaza apelurile de functii (in varf e intotdeauna functia curenta)

Heap-ul se extinde in jos, stack-ul se extinde in sus

Daca devin prea mari, vor incepe sa interfereze una cu cealalta

Sursele si constantele sunt zona pura a unui proces → se refolosesc in memorie

Cand se face un fork ce se duplica este contextul, variabilele, heap-ul si stack-ul se copiaza in memorie

La thread: contextul ramane acelasi

Sursele raman aceleasi, constantele raman aceleasi

Heap-ul ramane acelasi

Stack, stackT → Cand se creeaza un thread, se aloca din heap un buffer care sa serveasca pe post de stack thread-ului proaspat creat

stackT → stack-ul specific thread-ului proaspat creat

Thread-ul are acces la toata memoria pentru ca este aceeasi

Mai eficienta

```
void* f(void* a) {
        int i, k = 0;
        for(int i = 0; i < 1000*1000; i++) {
            k++;
        }
        pthread_mutex_lock(&m);
        n += k;
        pthread_mutex_unlock(&m);
```

```
}
```

Read-Write lock

```
#include <stdio.h>
#include <pthread.h>
#include <time.h>
struct timespec start;
int x = 0;
int y = 0;
int go() {
	struct timespec now;
	clock_gettime(CLOCK_MONOTONIC, &now);
	return ((now.tv_sec - start.tv_sec)*1000.0+
		(now.tv_nsec - start.tv_nsec)/1000.0/1000.0) < 5000;
}
void idle() {
	struct timespec t;
	t.tv_sec = 0;
	t.tv_nsec = 10*1000*1000;
	nanosleep(&t, NULL);
}
void* writer(void* a) {
	while(go()) {//Ruleaza in bucla timp de 5 secunde
		idle();//Ia o pauza de 10 milisecunde
		x++;//Mareste x
		y++;//Mareste y
	}
	return NULL;
}
```

//Problema de concurenta: Intre linia x++ si linia y++ exista o fractiune de microsecunda in care x este mai mare decat y. Pentru ca nu exista un Mutex care sa le lege strans, ceilalti cititori se pot uita la date exact in acea fereastra.

```
void* reader(void* a) {
	int errors = 0, reads = 0;//Contoare locale pentru fiecare cititor
	while(go()) {//Ruleaza in bucla timp de 5 secunde
		reads++;//Numara cate citiri a reusit sa faca
		if (x != y) {
			errors++;//Numara eroarea de sincronizare
		}
	}
	printf("Reads: % 8d Errors: % 8d\n", reads, errors);
	return NULL;
}
int main(int argc, char** argv) {
	pthread_t t[20];//Un array pentru 20 de thread-uri in total
	clock_gettime(CLOCK_MONOTONIC, &start);//Salvam momentul de pornire a programului
	//Cream primele 2 thread-uri ca fiind scriitori (pe pozitiile 0 si 1 in vector)
	pthread_create(&t[0], NULL, writer, NULL);
	pthread_create(&t[1], NULL, writer, NULL);
	for (int i = 2; i < 20; i++) {
		pthread_create(&t[i], NULL, reader, NULL);
	}
//Asteptam ca toate cele 20 de thread-uri sa termine (dupa ce trec cele 5 secunde)
	for (int i = 0; i < 20; i++) {
		pthread_join(t[i], NULL);
	}
	return 0;
}
```

```
Reads:  100866677 Errors:       12
Reads:  109952347 Errors:       22
Reads:  99917763 Errors:       27
Reads:  124945475 Errors:       21
```

Reads: 110350882 Errors: 18

Reads: 108599781 Errors: 17

Reads: 102118768 Errors: 21

Reads: 107887619 Errors: 19

Reads: 107094206 Errors: 16

Reads: 104774570 Errors: 15

Reads: 102926724 Errors: 22

Reads: 108941431 Errors: 19

Reads: 113953269 Errors: 11

Reads: 101463207 Errors: 14

Reads: 101926860 Errors: 11

Reads: 105622551 Errors: 23

Reads: 100371969 Errors: 17

Reads: 116194836 Errors: 25

Rezolvare erori:

```
#include <stdio.h>
#include <pthread.h>
#include <time.h>
pthread_mutex_t m;
struct timespec start;
int x = 0;
int y = 0;
int go() {
	struct timespec now;
	clock_gettime(CLOCK_MONOTONIC, &now);
	return ((now.tv_sec - start.tv_sec)*1000.0+
		(now.tv_nsec - start.tv_nsec)/1000.0/1000.0) < 5000;
}
void idle() {
	struct timespec t;
```

```
        t.tv_sec = 0;
        t.tv_nsec = 10*1000*1000;
        nanosleep(&t, NULL);
}
void* writer(void* a) {
        while(go()) {
                idle();
                pthread_mutex_lock(&m);
                x++;
                y++;
                pthread_mutex_unlock(&m);
        }
        return NULL;
}
void* reader(void* a) {
        int errors = 0, reads = 0;
        while(go()) {
                reads++;
                pthread_mutex_lock(&m);
                if (x != y) {
                        errors++;
                }
               pthread_mutex_unlock(&m);
        }
        printf("Reads: % 8d Errors: % 8d\n", reads, errors);
        return NULL;
}
int main(int argc, char** argv) {
        pthread_t t[20];
        clock_gettime(CLOCK_MONOTONIC, &start);
```

```
	pthread_mutex_init(&m, NULL);
	pthread_create(&t[0], NULL, writer, NULL);
	pthread_create(&t[1], NULL, writer, NULL);
	for (int i = 2; i < 20; i++) {
		pthread_create(&t[i], NULL, reader, NULL);
	}
	for (int i = 0; i < 20; i++) {
		pthread_join(t[i], NULL);
	}
	pthread_mutex_destroy(&m);
	return 0;
}
```

Cu Read-Write Lock

```
#include <stdio.h>
#include <pthread.h>
#include <time.h>
pthread_rwlock_t rwl;
struct timespec start;
int x = 0;
int y = 0;
int go() {
	struct timespec now;
	clock_gettime(CLOCK_MONOTONIC, &now);
	return ((now.tv_sec - start.tv_sec)*1000.0+
		(now.tv_nsec - start.tv_nsec)/1000.0/1000.0) < 5000;
}
void idle() {
	struct timespec t;
	t.tv_sec = 0;
	t.tv_nsec = 10*1000*1000;
```

```
        nanosleep(&t, NULL);
}
void* writer(void* a) {
        while(go()) {
                idle();
                pthread_rwlock_wrlock(&rwl);
                x++;
                y++;
                pthread_rwlock_unlock(&rwl);
        }
        return NULL;
}
void* reader(void* a) {
        int errors = 0, reads = 0;
        while(go()) {
                reads++;
                pthread_rwlock_rdlock(&rwl);
                if (x != y) {
                        errors++;
                }
                pthread_rwlock_unlock(&rwl);
   }
        printf("Reads: % 8d Errors: % 8d\n", reads, errors);
        return NULL;
}
int main(int argc, char** argv) {
        pthread_t t[20];
        clock_gettime(CLOCK_MONOTONIC, &start);
        pthread_rwlock_init(&rwl, NULL);
        pthread_create(&t[0], NULL, writer, NULL);
```

```
    pthread_create(&t[1], NULL, writer, NULL);
    for (int i = 2; i < 20; i++) {
        pthread_create(&t[i], NULL, reader, NULL);
    }
    for (int i = 0; i < 20; i++) {
        pthread_join(t[i], NULL);
    }
    pthread_rwlock_destroy(&rwl);

    return 0;
}
```

Curs 9

Variabile Conditionale

Write-Lock se comporta la fel ca Mutex-Lock => doar un thread write lock o singura data

Read-Lock lasa oricate thread-uri sa treaca prin el, cate vreme nu a trecut niciunul prin write lock

Sau oricati read-eri, sau doar un singur writer

Variabile conditionale

- Pot sa fac un thread sa astepte, pana cand alt thread il trezeste
- Variabilele conditionale functioneaza in conjunctie cu un mutex => nu vom avea variabile conditionale fara sa avem un mutex langa ea

//Thread-ul fa printeaza mesaje de la A0 la A9 si trmite semnale de trezire //Thread-ul fb intra in modul de asteptare si nu are voie sa printeze B pana cand nu primeste un semnal de trezire de la fa

```
#include <stdio.h>                                  #include <pthread.h>
pthread_mutex_t m;//mutex-ul obligatoriu (variabilele de conditie functioneaza doar cu un mutex)

pthread_cond_t c;//variabila de conditie (canalul prin care se dau semnalele)


void* fa(void* a) {
    (void)a;
    for (int i = 0; i < 10; i++) {
        printf("A%d\n", i);//printeaza textul (A0, A1, etc.)
        pthread_mutex_lock(&m);//incuie mutex-ul
```

```
        pthread_cond_signal(&c);//trezeste variabila conditionala
        pthread_mutex_unlock(&m);//descuie mutex-ul ca celalalt sa poata pleca
    }
    return NULL;
}

void* fb(void* a) {
    (void)a;
    pthread_mutex_lock(&m);//incuie mutex-ul
    pthread_cond_wait(&c, &m);//thread-ul se opreste din rulare si asteapta trezirea=>starea de somn
    pthread_mutex_unlock(&m);
    printf("B\n");//printeaza textul
    return NULL;
}

int main(int argc, char** argv) {
    (void)argc;
    (void)argv;
    pthread_t t[2];
    pthread_mutex_init(&m, NULL);
    pthread_cond_init(&c, NULL);
    pthread_create(&t[0], NULL, fa, NULL);
    pthread_create(&t[1], NULL, fb, NULL);
    for (int i = 0; i < 2; i++) {
        pthread_join(t[i], NULL);
    }
    pthread_mutex_destroy(&m);
    pthread_cond_destroy(&c);
    return 0;
}
```

```
#include <stdio.h>
#include <pthread.h>
#include <time.h>
struct timespec start;
pthread_mutex_t m;
pthread_cond_t c;
int honey = 10;
int go() {
                        struct timespec now;
                        clock_gettime(CLOCK_MONOTONIC, &now);
                        return ((now.tv_sec - start.tv_sec)*1000.0
                        (now.tv_nsec - start.tv_nsec)/1000.0/1000.0) < 5000;
}
void* bear(void* a) {
                        (void)a;
                        while(go()) {
                        pthread_mutex_lock(&m);
                         if (honey > 0) {
                                honey--;//timp de 5 secunde, ursul va tot manca miere
          }
else {
                        pthread_cond_signal(&c);
                        printf("!");//daca se termina mierea ursul il trezeste
             //pe fermier
             }
             pthread_mutex_unlock(&m);
       }
       pthread_mutex_lock(&m);
       pthread_cond_signal(&c);
```

```
	pthread_mutex_unlock(&m);
	return NULL;
}
void* bee(void* a) {
	(void)a;
	while(go()) {
		pthread_mutex_lock(&m);
		honey++;//timp de 5 secunde, albina va tot adauga miere
		pthread_mutex_unlock(&m);
	}
	return NULL;
}
void* farmer(void* a) {
	(void)a;
	while(go()) {
		pthread_mutex_lock(&m);
		while(go() && honey > 0) {
			printf("Zzzzzz...\n\n");
			pthread_cond_wait(&c, &m);
		}
		printf("OK\n\n");
		honey += 100;//cand este trezit, fermierul se duce si adauga
			//100 de unuitati de miere
		pthread_mutex_unlock(&m);
	}
	return NULL;
}
int main(int argc, char** argv) {
	(void)argc;
	(void)argv;
```

```
	pthread_t t[21];
	clock_gettime(CLOCK_MONOTONIC, &start);
	pthread_mutex_init(&m, NULL);
	pthread_cond_init(&c, NULL);
	pthread_create(&t[0], NULL, farmer, NULL);
	for (int i = 1; i < 20; i = i+2) {
		pthread_create(&t[i], NULL, bee, NULL);//10 albine
		pthread_create(&t[i+1], NULL, bear, NULL);//10 ursi
	}
	for (int i = 0; i < 21; i++) {
		pthread_join(t[i], NULL);
	}
	pthread_mutex_destroy(&m);
	pthread_cond_destroy(&c);
	return 0;
```

Output:

Zzzzzz...

!Zzzzzz...

!!!!!!!!!!!!!!!!!!!!OK

Zzzzzz...

!!!!!!!!!!!!!!!!!!!

!!!

```
void* f(void* a) {
	int p = *(int*)a;
//Salvam valoarea din pointer in variabila locala p (va fi 0, 1, 2...9)
	free(a);
	idle(500*p);
//Fiecare thread doarme diferit:
//Thread 0 → 500*0 = 0ms
//Thread 1 → 500*1 = 500 ms (0.5 secunde)
...
//Thread 9 → 500*9 = 4500 ms (4.5 secunde)
	printf("Reached barrier after % 3.1lf seconds\n", elapsed());
	pthread_barrier_wait(&b); //Thread-ul se opreste aici si asteapta la bariera
	printf("Moving on after % 1.1lf seconds\n", elapsed());
	return NULL;
}

int main(int argc, char** argv) {
	(void)argc;
	(void)argv;
	int* p;
	pthread_t t[21];
	clock_gettime(CLOCK_MONOTONIC, &start);
//Initializarea barierei
//Al treile argument (10) ii spune sistemului ”Bariera se va deschide doar cand 10 thread-uri vor rula
//’barrier_wait’”
	pthread_barrier_init(&b, NULL, 10);
	for (int i = 0; i < 10; i++) {
	  p = malloc(sizeof(int));
	    *p = i;
```

```
        pthread_create(&t[i], NULL, f, p);//Pornim cele 10 thread-uri
    }
    for (int i = 0; i < 10; i++) {
        pthread_join(t[i], NULL);
    }
    pthread_barrier_destroy(&b);
    return 0;
}
```

```
alexandra@LAPTOP-ALLE:~/threads$ ./prog19
Reached barrier after  0.0 seconds
Reached barrier after  0.5 seconds
Reached barrier after  1.0 seconds
Reached barrier after  1.5 seconds
Reached barrier after  2.0 seconds
Reached barrier after  2.5 seconds
Reached barrier after  3.0 seconds
Reached barrier after  3.5 seconds
Reached barrier after  4.0 seconds
Reached barrier after  4.5 seconds
Moving on after  4.5 seconds
Moving on after  4.5 seconds
Moving on after  4.5 seconds
Moving on after  4.5 seconds
Moving on after  4.5 seconds
Moving on after  4.5 seconds
Moving on after  4.5 seconds
Moving on after  4.5 seconds
Moving on after  4.5 seconds
Moving on after  4.5 seconds
```

Semafoare

- Semaforul este un mecanism care controleaza aglomeratia
- Permite numai unui numar predefinit de thread-uri sa intre in regiunea critica

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
sem_t sem;
void idle() {
	struct timespec t;
	t.tv_sec = 0;
	t.tv_nsec = 10*1000*1000;
	nanosleep(&t, NULL);
}

void* f(void* a) {
	(void)a;
	int v;
	sem_wait(&sem);
	sem_getvalue(&sem, &v);
	printf("%d ", v);
	idle();
	sem_post(&sem);
	return NULL;
}

int main(int argc, char** argv) {
	(void)argc;
	(void)argv;
	pthread_t t[50];
	sem_init(&sem, 0, 3);
	for (int i = 0; i < 50; i++) {
```

```
        pthread_create(&t[i], NULL, f, NULL);
    }
    for (int i = 0; i < 50; i++) {
        pthread_join(t[i], NULL);
    }
    sem_destroy(&sem);
    return 0;
}
```

```
alexandra@LAPTOP-ALLE:~/threads$ ./prog20
2 1 0 1 1 0 1 0 0 0 0 0 1 0 0 2 1 0 2 1 0 0 1 0 2 1 0 2 1 0 2 1 0 1 0 0 1 0 aaalexandra@LAPTOP-ALLE:~/threads$ ./prog20
2 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
```